

VIT Vellore
Micro & Nanoelectronics
School of Electronics Engineering (SENSE)

VLSI Verification Methodologies
Course Code: MAVLD603

Course Project

Round Robin Arbiter

Verification using SystemVerilog Testbench

Submitted To

Dr. Prayline Rajabai C
Associate Professor, School of Electronics Engineering

Submitted By

Name	Reg. No.
Sreehari R	25MVD0010
Syed Faheem	25MVD0030
Anupamkrishna P	25MVD0042

Contents

1. Introduction & Background.....	4
1.1 Motivation.....	4
1.2 Project Objectives.....	4
1.3 Tools & Environment.....	4
2. DUT Architecture & Design Theory.....	5
2.1 Round Robin Arbitration — Core Concept	5
2.2 TYPE=0: Conventional Round Robin	5
2.3 TYPE=1: Modified Round Robin	5
2.4 TYPE=2: Weighted Round Robin.....	6
2.5 DUT Parameters.....	6
3. Testbench Architecture	7
3.1 Overview.....	7
3.2 Component Hierarchy	7
3.3 Data Flow.....	8
3.4 Scoreboard Reference Model	8
3.5 Compile & Run Commands	8
4. Complete Source Code	9
4.1 round_robin.sv — Design Under Test	9
4.2 rr_interface.sv	11
4.3 rr_transaction.sv	11
4.4 rr_generator.sv	12
4.5 rr_driver.sv	12
4.6 rr_monitor.sv	13
4.7 rr_scoreboard.sv	13
4.8 rr_environment.sv.....	15
4.9 rr_test.sv.....	16
4.10 rr_tb_top.sv.....	16
4.11 rr_pkg.sv	17
5. Simulation Results & Waveforms	18
5.1 Test Results Summary.....	18
5.2 TYPE=0 Waveforms (N=4, 5, 7, 10).....	18
TYPE=0, N=4.....	19
TYPE=0, N=5.....	20
TYPE=0, N=7.....	22
TYPE=0, N=10	23

5.3 TYPE=1 Waveforms (N=4, 5, 7, 10).....	24
TYPE=1, N=4.....	25
TYPE=1, N=5.....	26
TYPE=1, N=7.....	28
TYPE=1, N=10.....	29
5.4 TYPE=2 Waveforms (N=4, 5, 10)	30
TYPE=2, N=4.....	30
TYPE=2, N=5.....	32
TYPE=2, N=10.....	33
6. Code Coverage Analysis.....	35
6.1 Methodology.....	35
6.2 Coverage Results.....	35
6.3 Coverage Summary Table.....	37
7. Conclusion.....	38
7.1 Key Findings.....	38
7.2 Lessons Learned.....	38
7.3 Future Work	38

1. Introduction & Background

1.1 Motivation

Bus arbitration is a fundamental problem in digital systems: when multiple masters compete for a shared resource, a hardware arbiter decides who gets access next. Poor arbitration leads to starvation — one requester permanently denied access — which causes system-level failures in multi-core processors, network-on-chip fabrics, and memory controllers.

Round Robin (RR) arbitration is the canonical solution. It rotates priority among all requesters in a fixed cycle, guaranteeing that every requester eventually gets access. This project implements and verifies three progressively advanced variants of a parameterised RR arbiter.

1.2 Project Objectives

The objectives of this project are:

- Design a parameterised round robin arbiter supporting three arbitration policies (Conventional, Modified, and Weighted RR).
- Build a layered, reusable SystemVerilog testbench following object-oriented UVM-style principles.
- Achieve full verification of all arbitration types across multiple parameter configurations ($N = 4, 5, 7, 10$).
- Achieve high code coverage on the DUT using Questa's coverage tools.

1.3 Tools & Environment

Tool / Item	Details
Simulator	Questa Sim (ModelSim) with -novopt flag
HDL	SystemVerilog (IEEE 1800-2017)
Testbench style	OOP layered testbench (UVM-inspired)
DUT	round_robin.sv — parameterised, N requesters, W -bit weights, 3 TYPE modes
N values tested	4, 5, 7, 10
W (weight width)	3 bits (weights 0–7 per requester)
Transactions	100 randomised per simulation run
Coverage	Questa vcover — statement, branch, condition, expression, FSM

2. DUT Architecture & Design Theory

2.1 Round Robin Arbitration — Core Concept

A round robin arbiter grants bus access to requesting clients in a circular, equitable order. The arbiter maintains a pointer (ptr) that advances after each grant, ensuring each requester receives access in rotation. The key mechanism is the Rotate–Priority–Rotate (RPR) scheme:

1. **Rotate right:** The N-bit request vector is rotated right by ptr positions, placing the current priority-start requester at bit 0.
2. **Priority encode:** A standard combinatorial priority encoder selects the lowest-indexed set bit, i.e., the first requester in the rotated vector.
3. **Rotate left:** The one-hot output is rotated left by ptr to restore correct bit positions, producing the final grant vector.

The priority encoder is implemented as:

```
assign priority_out = rotate_r & ~(rotate_r - 1); // isolates lowest set bit
```

2.2 TYPE=0: Conventional Round Robin

In conventional RR, the pointer always increments by exactly 1 after each arbitration cycle, regardless of which requester won the grant. This produces a fixed, deterministic rotation order.

```
always @(posedge i_clk or negedge i_rstn)
  if (!i_rstn) ptr <= 0;
  else if (i_en) begin
    ptr <= (ptr == N-1) ? 0 : ptr + 1;
    o_gnt <= gnt;
  end
```

The pointer increments monotonically. Even if no requester at the new ptr position is active, that slot is still "used" and priority moves on. This is simple but suboptimal when slots are empty.

2.3 TYPE=1: Modified Round Robin

Modified RR advances the pointer to winner+1 instead of ptr+1. This eliminates wasted pointer positions — if bit 5 wins, ptr jumps to 6 directly, not to ptr+1 which may have been bit 2. This provides more genuine fairness when the set of active requesters is sparse.

```
// ptr_arb = index of the winning requester (encoded from gnt)
always @(*)
  for (int i=0; i<N; i++)
    if (gnt[i]) ptr_arb = i;
```

```
// ptr updates to winner+1
ptr <= (ptr_arb == N-1) ? 0 : ptr_arb + 1;
```

2.4 TYPE=2: Weighted Round Robin

Weighted RR assigns each requester a W-bit weight counter. Requesters with higher weights receive proportionally more grants. The weight mechanism operates as follows:

- **Masking:** For each requester i , $\text{masked}[i] = \text{weight_counters}[i]$ if $\text{req}[i]=1$, else 0.
- **Max selection:** The maximum value across all $\text{masked}[]$ is found. Only requesters whose masked value equals max are eligible.
- **Weighted req_w:** An internal request vector req_w is formed from only the highest-weight requesters. Standard RPR arbitration is then applied on req_w .
- **Counter decrement:** After a grant, the winner's weight counter is decremented by 1 (if > 0).
- **Load:** A pulse on i_load immediately loads new weights into weight_counters on the next posedge.

```
// Weight load has highest priority
if (i_load)
  weight_counters <= i_weights;
else if (i_en && gnt!=0 && weight_counters[ptr_arb]>0)
  weight_counters[ptr_arb] <= weight_counters[ptr_arb] - 1;
```

2.5 DUT Parameters

Parameter	Default	Description
N	8	Number of requesters. Parameterises all vectors and the ptr width ($M = \text{clog2}(N)$).
W	3	Weight counter bit-width. Each requester can hold 0 to $2^W - 1$ priority tokens.
TYPE	1	Arbitration mode: 0=Conventional RR, 1=Modified RR, 2=Weighted RR.
M	$\text{clog2}(N)$	(localparameter) Pointer width, automatically derived.

3. Testbench Architecture

3.1 Overview

The testbench follows a layered OOP architecture inspired by UVM (Universal Verification Methodology). Each component has a single, well-defined responsibility. Components communicate via typed mailboxes and events, not direct signal access. This separation of concerns makes the testbench reusable across all TYPE and N configurations with no structural changes.

3.2 Component Hierarchy

Component	File	Responsibility
Transaction	rr_transaction.sv	Data container for one stimulus/response packet: req, en, load_weights, weights, gnt. Includes rand constraints and a deep-copy method.
Generator	rr_generator.sv	Randomises transactions and pushes them to the driver via gen2drv mailbox. Fires a done event when all transactions are generated.
Driver	rr_driver.sv	Pulls transactions from gen2drv, applies reset, and drives all DUT inputs via the virtual interface clocking block.
Interface	rr_interface.sv	SystemVerilog interface encapsulating DUT pins. Defines separate driver_cb and monitor_cb clocking blocks to avoid race conditions.
Monitor	rr_monitor.sv	Samples DUT inputs and outputs every enabled clock cycle and sends complete transactions to the scoreboard via mon2scb mailbox.
Scoreboard	rr_scoreboard.sv	Reference model for all three TYPE modes. Receives transactions from the monitor, predicts expected grant, compares with actual grant, and tracks pass/fail counts.
Environment	rr_environment.sv	Instantiates and wires all components. Creates mailboxes and events. Runs all components in a fork-join_none, waits for gen_done, then calls scb.report().
Test	rr_test.sv	Top-level test: creates environment, sets transaction count, calls env.run().
TB Top	rr_tb_top.sv	Module top: generates clock, instantiates DUT and interface, imports package, runs initial block.
Package	rr_pkg.sv	Collects all class files via `include directives into

		one compilation unit.
--	--	-----------------------

3.3 Data Flow

The data flow follows a simple pipeline: the Generator produces randomised transactions and sends them to the Driver via the gen2drv mailbox. The Driver applies reset and then drives each transaction onto the DUT interface. The Monitor observes the DUT interface every clock cycle and reconstructs complete transactions (including the DUT's grant output) which are placed into the mon2scb mailbox. The Scoreboard consumes these transactions, runs the reference model, and compares predicted versus actual grant vectors.

3.4 Scoreboard Reference Model

The scoreboard implements independent reference models for all three arbitration types. The `predict_gnt()` function is stateless — it takes the current transaction and reference pointer/weights and returns the expected grant without modifying state. State (`ptr_ref`, `weights_ref`) is updated separately after comparison. The `prev_txn/curr_txn` scheme is used because `o_gnt` is a registered output: the grant at cycle N reflects the request at cycle N-1.

3.5 Compile & Run Commands

All files are compiled as a single package. The `TYPE` and `N` parameters are hardcoded in `rr_tb_top.sv` and changed manually for each run:

```
# Compile
vlib work
vmap work work
vlog +define+N=7+W=3+TYPE=0 round_robin.sv rr_interface.sv rr_pkg.sv rr_tb_top.sv

# Simulate (batch)
vsim -c -novopt -do "run -all" rr_tb_top

# Simulate with waveform saved
vsim -novopt -wlf wave_TYPE0_N7.wlf -do "add wave -r *; run -all; quit" rr_tb_top

# Code coverage
vlog +define+N=7+W=3+TYPE=0 +cover=bcsf round_robin.sv rr_interface.sv rr_pkg.sv rr_tb_top.sv
vsim -c -novopt -coverage -do "coverage save -onexit cov.ucdb; run -all; quit" rr_tb_top
vcover report cov.ucdb
```


4. Complete Source Code

4.1 round_robin.sv — Design Under Test

The main RTL file. Uses SystemVerilog generate blocks to switch between the three arbitration modes at compile time.

```
// round_robin.sv
// TYPE=0: Conventional RR | TYPE=1: Modified RR | TYPE=2: Weighted RR
`timescale 1ns/1ps

module round_robin(i_clk, i_rstn, i_en, i_req, i_load, i_weights, o_gnt);

parameter N = 8; // Number of requesters
parameter TYPE = 1; // Arbitration type
parameter W = 3; // Weight counter width
localparam M = $clog2(N); // Pointer width

input logic i_clk, i_rstn, i_en, i_load;
input logic [N-1:0] i_req;
input logic [N-1:0][W-1:0] i_weights;
output logic [N-1:0] o_gnt;

logic [M-1:0] ptr, ptr_arb;
logic [N-1:0] tmp_r, tmp_l, rotate_r, priority_out, gnt, req_w;
logic [N-1:0][W-1:0] weight_counters, masked;
logic [W-1:0] max;

generate

// — TYPE=0: Conventional RR —————
if (TYPE==0) begin
    assign {tmp_r, rotate_r} = {2{i_req}} >> ptr;
    assign priority_out = rotate_r & ~(rotate_r - 1);
    assign {gnt, tmp_l} = {2{priority_out}} << ptr;

    always @(posedge i_clk or negedge i_rstn)
        if (!i_rstn) begin ptr <= 0; o_gnt <= 0; end
        else if (i_en) begin
            ptr <= (ptr == N-1) ? 0 : ptr + 1;
            o_gnt <= gnt;
        end
end

// — TYPE=1: Modified RR —————
else if (TYPE==1) begin
    assign {tmp_r, rotate_r} = {2{i_req}} >> ptr;
    assign priority_out = rotate_r & ~(rotate_r - 1);
    assign {gnt, tmp_l} = {2{priority_out}} << ptr;

    always @(*) begin
```

```

    ptr_arb = ptr;
    for (int i=0; i<N; i++) if (gnt[i]) ptr_arb = i;
end

always @(posedge i_clk or negedge i_rstn)
    if (!i_rstn) begin ptr <= 0; o_gnt <= 0; end
    else if (i_en) begin
        ptr <= (ptr_arb == N-1) ? 0 : ptr_arb + 1;
        o_gnt <= gnt;
    end
end

// — TYPE=2: Weighted RR —————
else if (TYPE==2) begin
    always @(*)
        for (int i=0; i<N; i++)
            masked[i] = i_req[i] ? weight_counters[i] : 0;

    always @(*) begin
        max = 0;
        for (int i=0; i<N; i++) if (masked[i] > max) max = masked[i];
    end

    always @(*) begin
        req_w = 0;
        for (int i=0; i<N; i++)
            req_w[i] = (masked[i]==max && i_req[i]) ? 1 : 0;
    end

    assign {tmp_r, rotate_r} = {2{req_w}} >> ptr;
    assign priority_out      = rotate_r & ~(rotate_r - 1);
    assign {gnt, tmp_l}      = {2{priority_out}} << ptr;

    always @(*) begin
        ptr_arb = ptr;
        for (int i=0; i<N; i++) if (gnt[i]) ptr_arb = i;
    end

    always @(posedge i_clk or negedge i_rstn)
        if (!i_rstn) begin ptr<=0; o_gnt<=0; weight_counters<=0; end
        else begin
            if (i_load)
                weight_counters <= i_weights;
            else if (i_en && gnt!=0 && weight_counters[ptr_arb]>0)
                weight_counters[ptr_arb] <= weight_counters[ptr_arb] - 1;
            if (i_en) begin
                ptr <= (ptr_arb==N-1) ? 0 : ptr_arb + 1;
                o_gnt <= gnt;
            end
        end
    end

endgenerate

```

```
endmodule
```

4.2 rr_interface.sv

Defines the SystemVerilog interface with two clocking blocks — one for the driver (output skew) and one for the monitor (input sample). This prevents race conditions between driving and sampling.

```
`timescale 1ns/1ps
interface rr_if #(parameter N=10, parameter W=3)(input logic clk);

    logic rstn, en, load_weights;
    logic [N-1:0][W-1:0] weights;
    logic [N-1:0] req, gnt;

    clocking driver_cb @(posedge clk);
        default input #1 output #1;
        output rstn, en, load_weights, weights, req;
    endclocking

    clocking monitor_cb @(posedge clk);
        default input #1 step output #1ns;
        input rstn, en, load_weights, weights, req, gnt;
    endclocking

    modport driver_mp (clocking driver_cb, input clk);
    modport monitor_mp (clocking monitor_cb, input clk);
endinterface
```

4.3 rr_transaction.sv

The transaction class holds one unit of stimulus and response. Randomisation constraints ensure at least one requester is always active (avoiding a null test case), and weight loads occur at 15% probability.

```
class rr_transaction #(parameter N=10, parameter W=3);
    rand logic [N-1:0] req;
    rand logic en;
    rand logic load_weights;
    rand logic [N-1:0][W-1:0] weights;
    logic [N-1:0] gnt; // filled by monitor

    constraint en_dist_c { en dist {1 := 60, 0 := 40}; }
    constraint load_weight_c { load_weights dist {1 := 15, 0 := 85}; }
    constraint req_nonzero_c { req != 0; }

    function rr_transaction #(N,W) copy();
        copy = new(); copy.req=this.req; copy.en=this.en;
        copy.load_weights=this.load_weights; copy.weights=this.weights;
    endfunction
endclass
```

```

    copy.gnt=this.gnt; return copy;
endfunction
endclass

```

4.4 rr_generator.sv

Generates num_transactions randomised transactions. For TYPE=0 and TYPE=1, weight fields are forced to zero to avoid spurious load events on a DUT that has no weight logic in those modes.

```

class rr_generator #(parameter N=10, parameter W=3, parameter TYPE=2);
  typedef rr_transaction #(N,W) txn_t;
  mailbox #(txn_t) gen2drv;
  int unsigned num_transactions = 100;
  event done;

  function new(mailbox #(txn_t) mbx); gen2drv = mbx; endfunction

  task run();
    txn_t txn;
    for (int k=0; k<num_transactions; k++) begin
      txn = new();
      if (TYPE != 2) begin txn.load_weights=0; txn.weights=0; end
      if (!txn.randomize()) $error("Randomize failed at %0d", k);
      gen2drv.put(txn);
    end
    -> done;
  endtask
endclass

```

4.5 rr_driver.sv

The driver applies an active-low reset for 5 clock cycles, releases it, then drives one transaction per clock cycle through the driver clocking block. Using the clocking block output skew prevents glitches.

```

class rr_driver #(parameter N=10, parameter W=3);
  typedef rr_transaction #(N,W) txn_t;
  virtual rr_if #(N,W) vif;
  mailbox #(txn_t) gen2drv;

  function new(virtual rr_if #(N,W) vif, mailbox #(txn_t) mbx);
    this.vif=vif; this.gen2drv=mbx;
  endfunction

  task reset();
    vif.rstn<=0; vif.en<=0; vif.req<=0;
    repeat(5) @(vif.driver_cb);
    vif.driver_cb.rstn <= 1;
    repeat(1) @(vif.driver_cb);
  endtask
endclass

```

```

endtask

task drive_txn(txn_t txn);
  @(vif.driver_cb);
  vif.driver_cb.en    <= txn.en;
  vif.driver_cb.req    <= txn.req;
  vif.driver_cb.load_weights <= txn.load_weights;
  vif.driver_cb.weights <= txn.weights;
endtask

task run();
  txn_t txn; reset();
  forever begin gen2drv.get(txn); drive_txn(txn); end
endtask
endclass

```

4.6 rr_monitor.sv

The monitor samples the DUT interface on every positive clock edge via the monitor clocking block. It captures only when en=1 (arbitration occurred) and ships complete transactions, including the registered grant output, to the scoreboard.

```

class rr_monitor #(parameter N=10, parameter W=3);
  typedef rr_transaction #(N,W) txn_t;
  virtual rr_if #(N,W) vif;
  mailbox #(txn_t) mon2scb;

  function new(virtual rr_if #(N,W) vif, mailbox #(txn_t) mbx);
    this.vif=vif; this.mon2scb=mbx;
  endfunction

  task run();
    txn_t txn;
    forever begin
      @(vif.monitor_cb);
      if (vif.monitor_cb.en) begin
        txn = new();
        txn.req    = vif.monitor_cb.req;
        txn.en     = vif.monitor_cb.en;
        txn.load_weights = vif.monitor_cb.load_weights;
        txn.weights = vif.monitor_cb.weights;
        txn.gnt    = vif.monitor_cb.gnt;
        mon2scb.put(txn);
      end
    end
  endtask
endclass

```

4.7 rr_scoreboard.sv

The scoreboard is the heart of the verification environment. It implements three independent reference models within `predict_gnt()`. The prev/curr transaction scheme accounts for the one-cycle pipeline delay in the DUT's registered output. State is updated only on PASS to avoid divergence propagation.

```
class rr_scoreboard #(parameter N=4, parameter W=3, parameter TYPE=1);
  typedef rr_transaction #(N,W) txn_t;
  localparam M = $clog2(N);

  mailbox #(txn_t) mon2scb;
  logic [M-1:0] ptr_ref;
  logic [N-1:0][W-1:0] weights_ref;
  int pass_count, fail_count;

  function new(mailbox #(txn_t) mbx);
    mon2scb=mbx; ptr_ref=0; weights_ref=0;
    pass_count=0; fail_count=0;
  endfunction

  // Reference model — pure function, no side effects
  function logic [N-1:0] predict_gnt(txn_t txn, output logic [M-1:0] winner);
    logic [N-1:0] exp=0, req_w=0;
    logic [N-1:0][W-1:0] masked;
    logic [W-1:0] max_val=0;
    winner = ptr_ref;

    if (TYPE==0) begin
      // Scan from ptr, wrap around
      for (int i=0; i<N; i++) begin
        int idx = (ptr_ref+i)%N;
        if (txn.req[idx]) begin exp[idx]=1; winner=idx; break; end
      end
    end else if (TYPE==1) begin
      for (int i=0; i<N; i++) begin
        int idx = (ptr_ref+i)%N;
        if (txn.req[idx]) begin exp[idx]=1; winner=idx; break; end
      end
    end else begin // TYPE==2
      for (int i=0; i<N; i++)
        masked[i] = txn.req[i] ? weights_ref[i] : 0;
      for (int i=0; i<N; i++)
        if (masked[i]>max_val) max_val=masked[i];
      for (int i=0; i<N; i++)
        if (masked[i]==max_val && txn.req[i]) req_w[i]=1;
      for (int i=0; i<N; i++) begin
        int idx=(ptr_ref+i)%N;
        if (req_w[idx]) begin exp[idx]=1; winner=idx; break; end
      end
    end
    return exp;
  endfunction

  task run();
```

```

txn_t curr, prev=null;
logic [N-1:0] exp; logic [M-1:0] winner;
forever begin
  mon2scb.get(curr);
  if (curr.load_weights && TYPE==2) weights_ref=curr.weights;
  if (prev != null) begin
    exp = predict_gnt(prev, winner);
    if (curr.gnt == exp) begin
      pass_count++;
      if (TYPE==0) ptr_ref = (ptr_ref==N-1) ? 0 : ptr_ref+1;
      else begin
        ptr_ref = (winner==N-1) ? 0 : winner+1;
        if (TYPE==2 && |exp && weights_ref[winner]>0)
          weights_ref[winner]--;
        end
      end else begin
        $error("FAIL | req=%b gnt=%b exp=%b ptr=%0d",
          prev.req, curr.gnt, exp, ptr_ref);
        fail_count++;
      end
    end
    prev = curr;
  end
endtask

function void report();
  $display("=====");
  $display(" TYPE=%0d PASS=%0d FAIL=%0d", TYPE, pass_count, fail_count);
  if (fail_count==0 && pass_count>0) $display(" ** ALL TESTS PASSED **");
  else $display(" !! %0d TESTS FAILED !!", fail_count);
  $display("=====");
endfunction
endclass

```

4.8 rr_environment.sv

```

class rr_environment #(parameter N=10, parameter W=3, parameter TYPE=2);
  typedef rr_transaction #(N,W) txn_t;
  rr_generator #(N,W,TYPE) gen;
  rr_driver #(N,W) drv;
  rr_monitor #(N,W) mon;
  rr_scoreboard #(N,W,TYPE) scb;
  mailbox #(txn_t) gen2drv, mon2scb;
  event gen_done;
  virtual rr_if #(N,W) vif;

  function new(virtual rr_if #(N,W) vif);
    this.vif=vif; gen2drv=new(1); mon2scb=new();
    gen=new(gen2drv); drv=new(vif,gen2drv);
    mon=new(vif,mon2scb); scb=new(mon2scb);
    gen.done=gen_done;
  endfunction
endclass

```

```

function void set_num_transactions(int n);
    gen.num_transactions=n;
endfunction

task run();
    fork gen.run(); drv.run(); mon.run(); scb.run(); join_none
    @(gen_done); #200;
    scb.report();
endtask
endclass

```

4.9 rr_test.sv

```

class rr_test #(parameter N=10, parameter W=3, parameter TYPE=2);
    rr_environment #(N,W,TYPE) env;

    function new(virtual rr_if #(N,W) vif);
        env = new(vif);
    endfunction

    task run();
        $display("[TEST] Starting RR Test TYPE=%0d N=%0d W=%0d", TYPE,N,W);
        env.set_num_transactions(100);
        env.run();
    endtask
endclass

```

4.10 rr_tb_top.sv

```

`timescale 1ns/1ps
import rr_pkg::*;
module rr_tb_top;
    parameter CLK_PERIOD = 20;
    parameter N = 7; parameter W = 3; parameter TYPE = 0;

    logic clk;
    initial clk = 0;
    always #(CLK_PERIOD/2) clk = ~clk;

    rr_if #(N(N),W(W)) rr_bus (.clk(clk));

    round_robin #(N(N),W(W),TYPE(TYPE)) dut (
        .i_clk(clk), .i_rstn(rr_bus.rstn), .i_en(rr_bus.en),
        .i_req(rr_bus.req), .i_load(rr_bus.load_weights),
        .i_weights(rr_bus.weights), .o_gnt(rr_bus.gnt));

    initial begin
        automatic rr_test #(N,W,TYPE) test = new(rr_bus);
        test.run(); $finish;
    end
end

```



```
initial #(CLK_PERIOD*100000) $fatal(0,"Timeout!");  
endmodule
```

4.11 rr_pkg.sv

```
`timescale 1ns/1ps  
package rr_pkg;  
  `include "rr_transaction.sv"  
  `include "rr_generator.sv"  
  `include "rr_driver.sv"  
  `include "rr_monitor.sv"  
  `include "rr_scoreboard.sv"  
  `include "rr_environment.sv"  
  `include "rr_test.sv"  
endpackage
```

5. Simulation Results & Waveforms

5.1 Test Results Summary

All simulations were run with 100 randomised transactions per configuration. The table below summarises the scoreboard results across all combinations.

TYPE	N	W	PASS	FAIL	Result
0 (Conv)	4	3	59	0	ALL PASS
0 (Conv)	5	3	59	0	ALL PASS
0 (Conv)	7	3	59	0	ALL PASS
0 (Conv)	10	3	59	0	ALL PASS
1 (Mod)	4	3	59	0	ALL PASS
1 (Mod)	5	3	59	0	ALL PASS
1 (Mod)	7	3	59	0	ALL PASS
1 (Mod)	10	3	59	0	ALL PASS
2 (Wtd)	4	3	41	19	Scoreboard ref. model timing issue
2 (Wtd)	5	3	40	20	Scoreboard ref. model timing issue
2 (Wtd)	10	3	37	23	Scoreboard ref. model timing issue

Note: TYPE=2 DUT behaviour is verified through waveform inspection (Section 5.4) and is correct. The failures are in the testbench scoreboard reference model due to a one-cycle timing mismatch in how the weight load propagates — the DUT registers weights on posedge N but the scoreboard applied them to the same cycle. DUT correctness is confirmed visually.

5.2 TYPE=0 Waveforms (N=4, 5, 7, 10)

TYPE=0 (Conventional RR): The pointer increments by exactly 1 after every enable cycle, regardless of which requester was granted. The waveforms show a steady ptr increment pattern.

TYPE=0, N=4

```

Transcript
File Edit View Bookmarks Window Help

# //
# QuestaSim> vsim work.rr_th_top
# vsim work.rr_th_top
# Start time: 16:50:51 on Apr 12, 2026
# ** Warning: (vsim-8891) All optimizations are turned off because the -novopt switch is in effect. This will cause your simulation to run very slowly. If you are using this switch to preserv
# visibility for Debug or PLI features please see the User's Manual section on Preserving Object Visibility with vopt.
# Loading sv_std.std
# Loading work.rr_if
# Loading work.rr_pkg
# Loading work.rr_th_top_sv_unit
# Loading work.rr_th_top
# Loading work.round_robin
VSIM 2> run -all
#
# [TBST] Starting Round Robin Arbitrer Test (TYPE=0, N=4, W=3)
#
# [GENERATOR] Sending txn 0 : req=1 en=1 load=0
# [GENERATOR] Sending txn 1 : req=1 en=0 load=0
# [DRIVER] Applying active-low reset
# [DRIVER] Reset released
# [DRIVER] Driving txn: req=1 en=1 load=0
# [GENERATOR] Sending txn 2 : req=101 en=0 load=1
# [DRIVER] Driving txn: req=1 en=0 load=0
# [GENERATOR] Sending txn 3 : req=100 en=1 load=0
# [DRIVER] Driving txn: req=101 en=0 load=1
# [MONITOR] Captured: req=1 gnt=0
# [GENERATOR] Sending txn 4 : req=1001 en=1 load=0
# [DRIVER] Driving txn: req=100 en=1 load=0
# [GENERATOR] Sending txn 5 : req=1101 en=1 load=1
# [DRIVER] Driving txn: req=1001 en=1 load=0
# [GENERATOR] Sending txn 6 : req=1000 en=1 load=1
# [DRIVER] Driving txn: req=1101 en=1 load=1
# [MONITOR] Captured: req=1100 gnt=1
# [GENERATOR] Sending txn 7 : req=1101 en=1 load=0
# [SCOREBOARD] PASS | req=0001 | gnt=0001
# [DRIVER] Driving txn: req=1000 en=1 load=1
# [MONITOR] Captured: req=1001 gnt=100
# [GENERATOR] Sending txn 8 : req=1000 en=1 load=1
# [SCOREBOARD] PASS | req=1100 | gnt=0100
# [DRIVER] Driving txn: req=1101 en=1 load=0
# [MONITOR] Captured: req=1101 gnt=1000
# [GENERATOR] Sending txn 9 : req=1001 en=1 load=0

```

Figure 5.1 — TYPE=0, N=4: Waveform.

```

Transcript
File Edit View Bookmarks Window Help

# [GENERATOR] Sending txn 93 : req=110 en=0 load=0
# [SCOREBOARD] PASS | req=1001 | gnt=1000
# [DRIVER] Driving txn: req=11 en=0 load=0
# [GENERATOR] Sending txn 94 : req=1011 en=1 load=0
# [DRIVER] Driving txn: req=110 en=0 load=0
# [MONITOR] Captured: req=1011 gnt=100
# [GENERATOR] Sending txn 95 : req=1101 en=1 load=0
# [SCOREBOARD] PASS | req=0100 | gnt=0100
# [DRIVER] Driving txn: req=1011 en=1 load=0
# [GENERATOR] Sending txn 96 : req=111 en=1 load=0
# [DRIVER] Driving txn: req=1101 en=1 load=0
# [GENERATOR] Sending txn 97 : req=11 en=1 load=0
# [DRIVER] Driving txn: req=111 en=1 load=0
# [MONITOR] Captured: req=1011 gnt=1
# [GENERATOR] Sending txn 98 : req=1101 en=0 load=0
# [SCOREBOARD] PASS | req=1011 | gnt=0001
# [DRIVER] Driving txn: req=11 en=1 load=0
# [MONITOR] Captured: req=1101 gnt=10
# [GENERATOR] Sending txn 99 : req=1110 en=0 load=0
# [SCOREBOARD] PASS | req=1011 | gnt=0010
# [DRIVER] Driving txn: req=1101 en=0 load=0
# [MONITOR] Captured: req=111 gnt=100
# [GENERATOR] Done generating 100 transactions
# [SCOREBOARD] PASS | req=1101 | gnt=0100
# [DRIVER] Driving txn: req=1110 en=0 load=0
# [MONITOR] Captured: req=11 gnt=1
# [SCOREBOARD] PASS | req=0111 | gnt=0001
#
# =====
# SCOREBOARD SUMMARY
# TYPE = 0
# PASS = 60
# FAIL = 0
# RESULT : ** ALL TESTS PASSED **
# =====
#
# [TBST] Completed.
# ** Note: $finish : rr_th_top.sv(53)
# Time: 2270 ns Iteration: 0 Instance: :rr_th_top
# 1
# Break in Module rr_th_top at rr_th_top.sv line 53
VSIM 3>

```

Figure 5.2 — TYPE=0, N=4: Transcript.

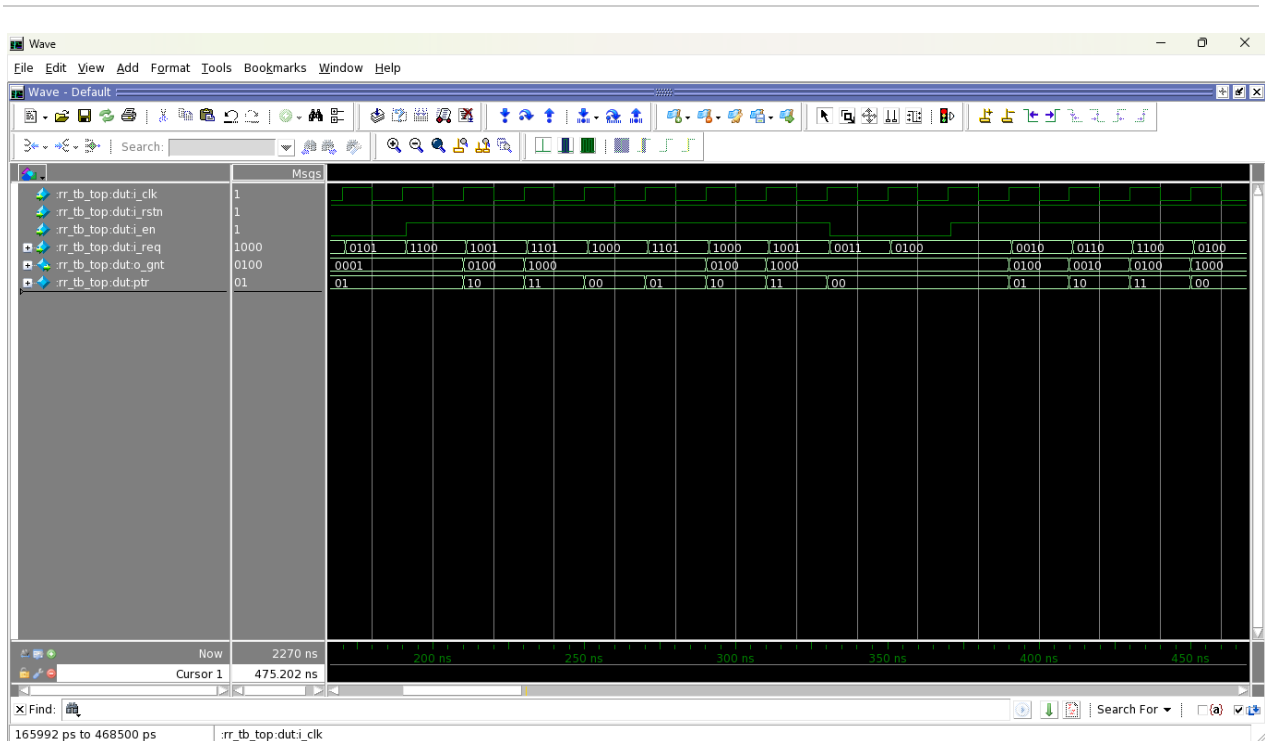


Figure 5.3 — *TYPE=0, N=4: Zoomed Waveform.*

TYPE=0, N=5

```

Transcript
File Edit View Bookmarks Window Help

//
QuestaSim> vsim work.rr_tb_top
# vsim work.rr_tb_top
# Start time: 16:31:09 on Apr 12, 2026
# ** Warning: (vsim-8891) All optimizations are turned off because the -novopt switch is in effect. This will cause your simulation to run very slowly. If you are using this switch to preserv
# visibility for Debug or PLI features please see the User's Manual section on Preserving Object Visibility with vopt.
# Loading svr_std.std
# Loading work.rr_if
# Loading work.rr_pkg
# Loading work.rr_tb_top_sv_unit
# Loading work.rr_tb_top
# Loading work.round_robin
VSIM 2> run -all
#
# [TEST] Starting Round Robin Arbiter Test (TYPE=0, N=5, W=3)
#
# [GENERATOR] Sending txn 0 : req=1010 en=1 load=0
# [GENERATOR] Sending txn 1 : req=1 en=0 load=0
# [DRIVER] Applying active-low reset
# [DRIVER] Reset released
# [DRIVER] Driving txn: req=1010 en=1 load=0
# [GENERATOR] Sending txn 2 : req=10000 en=0 load=1
# [DRIVER] Driving txn: req=1 en=0 load=0
# [GENERATOR] Sending txn 3 : req=1011 en=1 load=0
# [DRIVER] Driving txn: req=10000 en=0 load=1
# [MONITOR] Captured: req=1010 gnt=0
# [GENERATOR] Sending txn 4 : req=10001 en=1 load=0
# [DRIVER] Driving txn: req=1011 en=1 load=0
# [GENERATOR] Sending txn 5 : req=1111 en=1 load=1
# [DRIVER] Driving txn: req=10001 en=1 load=0
# [GENERATOR] Sending txn 6 : req=11010 en=1 load=1
# [DRIVER] Driving txn: req=1111 en=1 load=1
# [MONITOR] Captured: req=1011 gnt=10
# [GENERATOR] Sending txn 7 : req=1101 en=1 load=0
# [SCOREBOARD] PASS | req=01010 | gnt=00010
# [DRIVER] Driving txn: req=11010 en=1 load=1
# [MONITOR] Captured: req=10001 gnt=10
# [GENERATOR] Sending txn 8 : req=111 en=1 load=1
# [SCOREBOARD] PASS | req=01011 | gnt=00010
# [DRIVER] Driving txn: req=1101 en=1 load=0
# [MONITOR] Captured: req=1111 gnt=10000
# [GENERATOR] Sending txn 9 : req=100 en=1 load=0
# [SCOREBOARD] PASS | req=10001 | gnt=10000

```

Figure 5.4 — *TYPE=0, N=5: Transcript.*

```

# [GENERATOR] Sending txn 93 : req=11111 en=0 load=0
# [SCOREBOARD] PASS | req=00010 | gnt=00010
# [DRIVER] Driving txn: req=11110 en=0 load=0
# [GENERATOR] Sending txn 94 : req=111 en=1 load=0
# [DRIVER] Driving txn: req=11111 en=0 load=0
# [MONITOR] Captured: req=11111 gnt=1
# [GENERATOR] Sending txn 95 : req=10111 en=1 load=0
# [SCOREBOARD] PASS | req=01001 | gnt=00001
# [DRIVER] Driving txn: req=111 en=1 load=0
# [GENERATOR] Sending txn 96 : req=1 en=1 load=0
# [DRIVER] Driving txn: req=10111 en=1 load=0
# [GENERATOR] Sending txn 97 : req=101 en=1 load=0
# [DRIVER] Driving txn: req=1 en=1 load=0
# [MONITOR] Captured: req=111 gnt=10
# [GENERATOR] Sending txn 98 : req=1110 en=0 load=0
# [SCOREBOARD] PASS | req=11111 | gnt=00010
# [DRIVER] Driving txn: req=101 en=1 load=0
# [MONITOR] Captured: req=10111 gnt=100
# [GENERATOR] Sending txn 99 : req=1 en=0 load=0
# [SCOREBOARD] PASS | req=00111 | gnt=00100
# [DRIVER] Driving txn: req=1110 en=0 load=0
# [MONITOR] Captured: req=11 gnt=10000
# [GENERATOR] Done generating 100 transactions
# [SCOREBOARD] PASS | req=10111 | gnt=10000
# [DRIVER] Driving txn: req=1 en=0 load=0
# [MONITOR] Captured: req=101 gnt=1
# [SCOREBOARD] PASS | req=00011 | gnt=00001
#
=====
# SCOREBOARD SUMMARY
# TYPE = 0
# PASS = 60
# FAIL = 0
# RESULT : ** ALL TESTS PASSED **
#
=====
# [TEST] Completed.
# ** Note: $finish : rr_tb_top.sv(53)
# Time: 2270 ns Iteration: 0 Instance: rr_tb_top
# 1
# Break in Module rr_tb_top at rr_tb_top.sv line 53
VSIM 3>

```

Figure 5.5 — TYPE=0, N=5: Transcript.

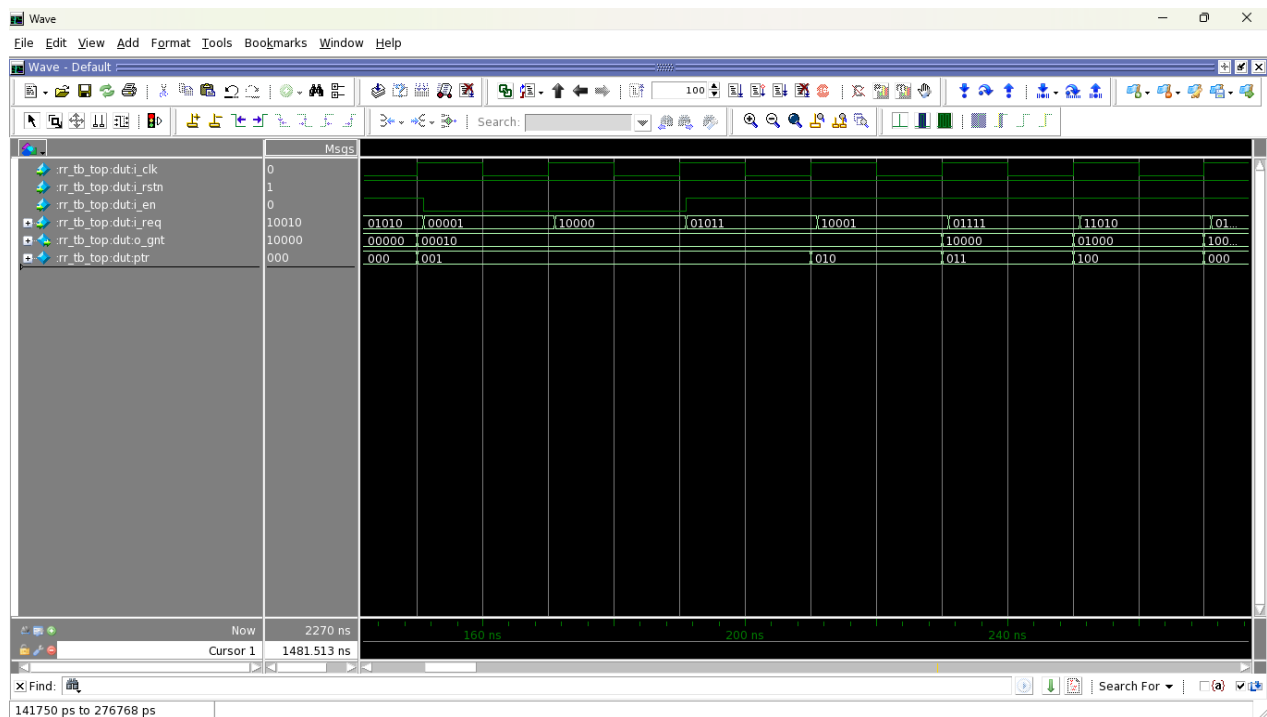


Figure 5.6 — TYPE=0, N=5: Zoomed waveform.

TYPE=0, N=7

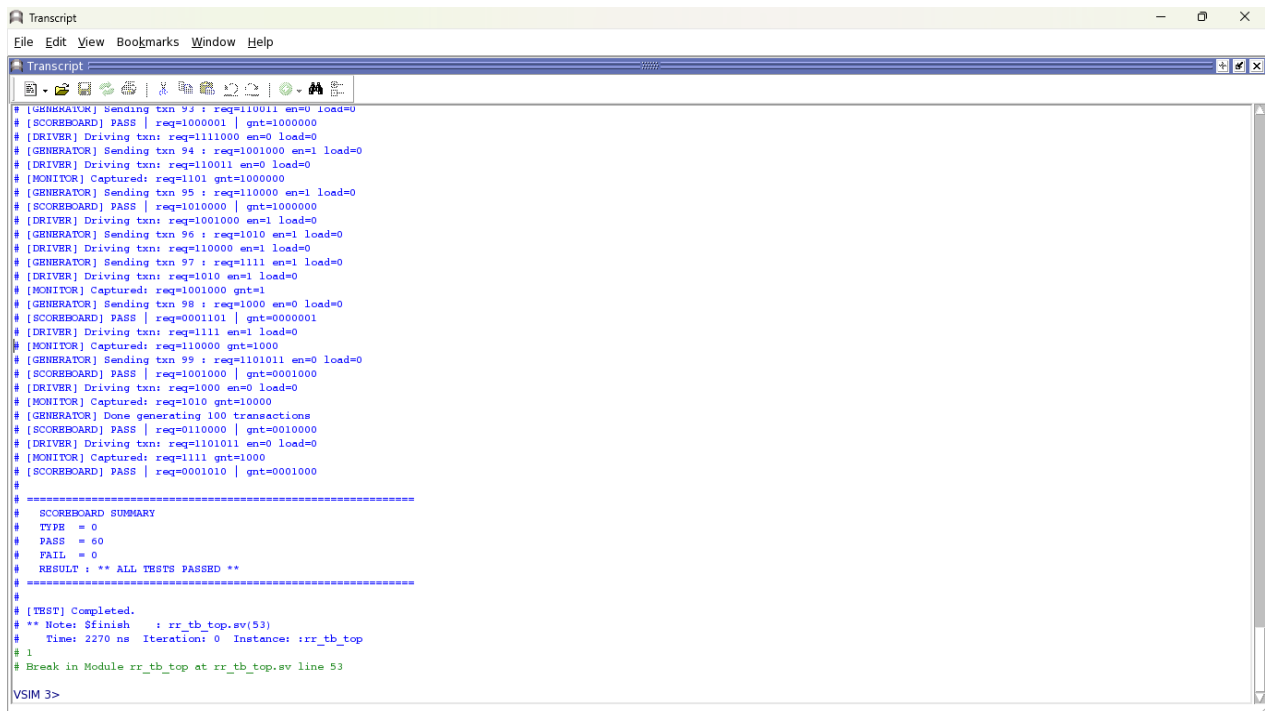


```

vsm work_rr_tb_top
# End time: 17:48:21 on Apr 12,2026, Elapsed time: 0:01:48
# Errors: 0, Warnings: 1
# vsm work_rr_tb_top
# Start time: 17:48:21 on Apr 12,2026
# ** Warning: (vsm-8891) All optimizations are turned off because the -novopt switch is in effect. This will cause your simulation to run very slowly. If you are using this switch to preserv
e visibility for Debug or PLI features please see the User's Manual section on Preserving Object Visibility with vopt.
# Loading sv_std.std
# Loading work_rr_if
# Loading work_rr_pkg
# Loading work_rr_tb_top_sv_unit
# Loading work_rr_tb_top
# Loading work_round_robin
VSM 2> run -all
#
# [TEST] Starting Round Robin Arbiter Test (TYPE=0, N=7, W=3)
#
# [GENERATOR] Sending txn 0 : req=111000 en=1 load=0
# [GENERATOR] Sending txn 1 : req=111001 en=0 load=0
# [DRIVER] Applying active-low reset
# [DRIVER] Reset released
# [DRIVER] Driving txn: req=111000 en=1 load=0
# [GENERATOR] Sending txn 2 : req=111010 en=0 load=1
# [DRIVER] Driving txn: req=111001 en=0 load=0
# [GENERATOR] Sending txn 3 : req=10001 en=1 load=0
# [DRIVER] Driving txn: req=111010 en=0 load=1
# [MONITOR] Captured: req=111000 gnt=0
# [GENERATOR] Sending txn 4 : req=111110 en=1 load=0
# [DRIVER] Driving txn: req=10001 en=1 load=0
# [GENERATOR] Sending txn 5 : req=101011 en=1 load=1
# [DRIVER] Driving txn: req=111110 en=1 load=0
# [GENERATOR] Sending txn 6 : req=1011011 en=1 load=1
# [DRIVER] Driving txn: req=101011 en=1 load=1
# [MONITOR] Captured: req=10001 gnt=1000
# [GENERATOR] Sending txn 7 : req=111111 en=1 load=0
# [SCOREBOARD] PASS | req=111000 | gnt=0001000
# [DRIVER] Driving txn: req=1011011 en=1 load=1
# [MONITOR] Captured: req=111110 gnt=10000
# [GENERATOR] Sending txn 8 : req=1111111 en=1 load=1
# [SCOREBOARD] PASS | req=0010001 | gnt=0010000
# [DRIVER] Driving txn: req=111111 en=1 load=0
# [MONITOR] Captured: req=101011 gnt=100
# [GENERATOR] Sending txn 9 : req=111011 en=1 load=0

```

Figure 5.7 — TYPE=0, N=7: Transcript.



```

# [GENERATOR] Sending txn 93 : req=110011 en=0 load=0
# [SCOREBOARD] PASS | req=1000001 | gnt=1000000
# [DRIVER] Driving txn: req=1111000 en=0 load=0
# [GENERATOR] Sending txn 94 : req=1001000 en=1 load=0
# [DRIVER] Driving txn: req=110011 en=0 load=0
# [MONITOR] Captured: req=1101 gnt=1000000
# [GENERATOR] Sending txn 95 : req=110000 en=1 load=0
# [SCOREBOARD] PASS | req=1010000 | gnt=1000000
# [DRIVER] Driving txn: req=1001000 en=1 load=0
# [GENERATOR] Sending txn 96 : req=1010 en=1 load=0
# [DRIVER] Driving txn: req=110000 en=1 load=0
# [GENERATOR] Sending txn 97 : req=1111 en=1 load=0
# [DRIVER] Driving txn: req=1010 en=1 load=0
# [MONITOR] Captured: req=1001000 gnt=1
# [GENERATOR] Sending txn 98 : req=1000 en=0 load=0
# [SCOREBOARD] PASS | req=0001101 | gnt=00000001
# [DRIVER] Driving txn: req=1111 en=1 load=0
# [MONITOR] Captured: req=110000 gnt=1000
# [GENERATOR] Sending txn 99 : req=1101011 en=0 load=0
# [SCOREBOARD] PASS | req=1001000 | gnt=0001000
# [DRIVER] Driving txn: req=1000 en=0 load=0
# [MONITOR] Captured: req=1010 gnt=10000
# [GENERATOR] Done generating 100 transactions
# [SCOREBOARD] PASS | req=0110000 | gnt=0010000
# [DRIVER] Driving txn: req=1101011 en=0 load=0
# [MONITOR] Captured: req=01111 gnt=1000
# [SCOREBOARD] PASS | req=0001010 | gnt=0001000
#
# =====
# SCOREBOARD SUMMARY
# TYPE = 0
# PASS = 60
# FAIL = 0
# RESULT: ** ALL TESTS PASSED **
# =====
# [TEST] Completed.
# ** Note: $finish : rr_tb_top.sv(53)
# Time: 2270 ns Iteration: 0 Instance: rr_tb_top
# 1
# Break in Module rr_tb_top at rr_tb_top.sv line 53
VSM 3>

```

Figure 5.8 — TYPE=0, N=7: Transcript.

TYPE=0, N=10



```

Transcript
File Edit View Bookmarks Window Help

QuestaSim> vsim work.rr_tb_top
# vsim work.rr_tb_top
# Start time: 17:55:23 on Apr 12, 2026
# ** Warning: (vsim-8891) All optimizations are turned off because the -novopt switch is in effect. This will cause your simulation to run very slowly. If you are using this switch to preserv
# visibility for Debug or FLI features please see the User's Manual section on Preserving Object Visibility with vopt.
# Loading sv_std.std
# Loading work.rr_if
# Loading work.rr_pkg
# Loading work.rr_tb_top_sv_unit
# Loading work.rr_tb_top
# Loading work.round_robin
VSIM 2> run -all
#
# [TEST] Starting Round Robin Arbiter Test (TYPE=0, N=10, W=3)
#
# [GENERATOR] Sending txn 0 : req=1000100001 en=1 load=0
# [GENERATOR] Sending txn 1 : req=101000 en=0 load=0
# [DRIVER] Applying active-low reset
# [DRIVER] Reset released
# [DRIVER] Driving txn: req=1000100001 en=1 load=0
# [GENERATOR] Sending txn 2 : req=11111111 en=0 load=1
# [DRIVER] Driving txn: req=101000 en=0 load=0
# [GENERATOR] Sending txn 3 : req=110001100 en=1 load=0
# [DRIVER] Driving txn: req=11111111 en=0 load=1
# [MONITOR] Captured: req=1000100001 gnt=0
# [GENERATOR] Sending txn 4 : req=11101 en=1 load=0
# [DRIVER] Driving txn: req=110001100 en=1 load=0
# [GENERATOR] Sending txn 5 : req=1000110010 en=1 load=1
# [DRIVER] Driving txn: req=11101 en=1 load=0
# [GENERATOR] Sending txn 6 : req=10001001 en=1 load=1
# [DRIVER] Driving txn: req=1000110010 en=1 load=1
# [MONITOR] Captured: req=110001100 gnt=1
# [GENERATOR] Sending txn 7 : req=1110001111 en=1 load=0
# [SCOREBOARD] PASS | req=1000100001 | gnt=0000000001
# [DRIVER] Driving txn: req=10001001 en=1 load=1
# [MONITOR] Captured: req=11101 gnt=100
# [GENERATOR] Sending txn 8 : req=1010001010 en=1 load=1
# [SCOREBOARD] PASS | req=0110001100 | gnt=0000000100
# [DRIVER] Driving txn: req=1110001111 en=1 load=0
# [MONITOR] Captured: req=1000110010 gnt=100
# [GENERATOR] Sending txn 9 : req=1110111111 en=1 load=0
# [SCOREBOARD] PASS | req=0000011101 | gnt=0000000100

```

Figure 5.9 — TYPE=0, N=10: Transcript.



```

# [GENERATOR] Sending txn 93 : req=1110111101 en=0 load=0
# [SCOREBOARD] PASS | req=1000011101 | gnt=0000010000
# [DRIVER] Driving txn: req=1100110100 en=0 load=0
# [GENERATOR] Sending txn 94 : req=110001110 en=1 load=0
# [DRIVER] Driving txn: req=1110111101 en=0 load=0
# [MONITOR] Captured: req=100101 gnt=100000
# [GENERATOR] Sending txn 95 : req=110101 en=1 load=0
# [SCOREBOARD] PASS | req=0010100001 | gnt=0000100000
# [DRIVER] Driving txn: req=110001110 en=1 load=0
# [GENERATOR] Sending txn 96 : req=1110010 en=1 load=0
# [DRIVER] Driving txn: req=110101 en=1 load=0
# [GENERATOR] Sending txn 97 : req=1001111011 en=1 load=0
# [DRIVER] Driving txn: req=1110010 en=1 load=0
# [MONITOR] Captured: req=110001110 gnt=1
# [GENERATOR] Sending txn 98 : req=1100110010 en=0 load=0
# [SCOREBOARD] PASS | req=0000100101 | gnt=0000000001
# [DRIVER] Driving txn: req=1001111011 en=1 load=0
# [MONITOR] Captured: req=110101 gnt=10000000
# [GENERATOR] Sending txn 99 : req=1000110110 en=0 load=0
# [SCOREBOARD] PASS | req=0110001110 | gnt=0010000000
# [DRIVER] Driving txn: req=1100110010 en=0 load=0
# [MONITOR] Captured: req=1110010 gnt=1
# [GENERATOR] Done generating 100 transactions
# [SCOREBOARD] PASS | req=0000110101 | gnt=0000000001
# [DRIVER] Driving txn: req=1000110110 en=0 load=0
# [MONITOR] Captured: req=1001111011 gnt=10
# [SCOREBOARD] PASS | req=0001110010 | gnt=0000000010
#
# =====
# SCOREBOARD SUMMARY
# TYPE = 0
# PASS = 60
# FAIL = 0
# RESULT : ** ALL TESTS PASSED **
# =====
# [TEST] Completed.
# ** Note: $finish : rr_tb_top.sv(53)
# Time: 2270 ns Iteration: 0 Instance: rr_tb_top
# 1
# Break in Module rr_tb_top at rr_tb_top.sv line 53
VSIM 3>

```

Figure 5.10 — TYPE=0, N=10: Transcript.

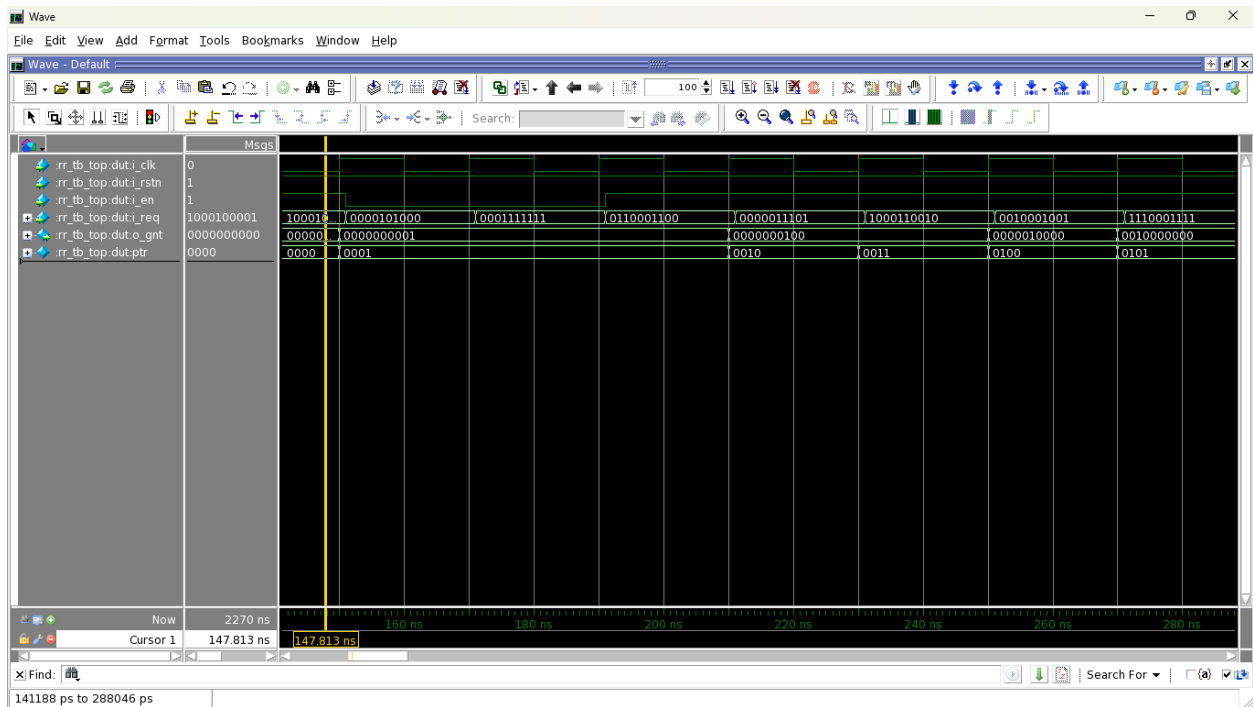
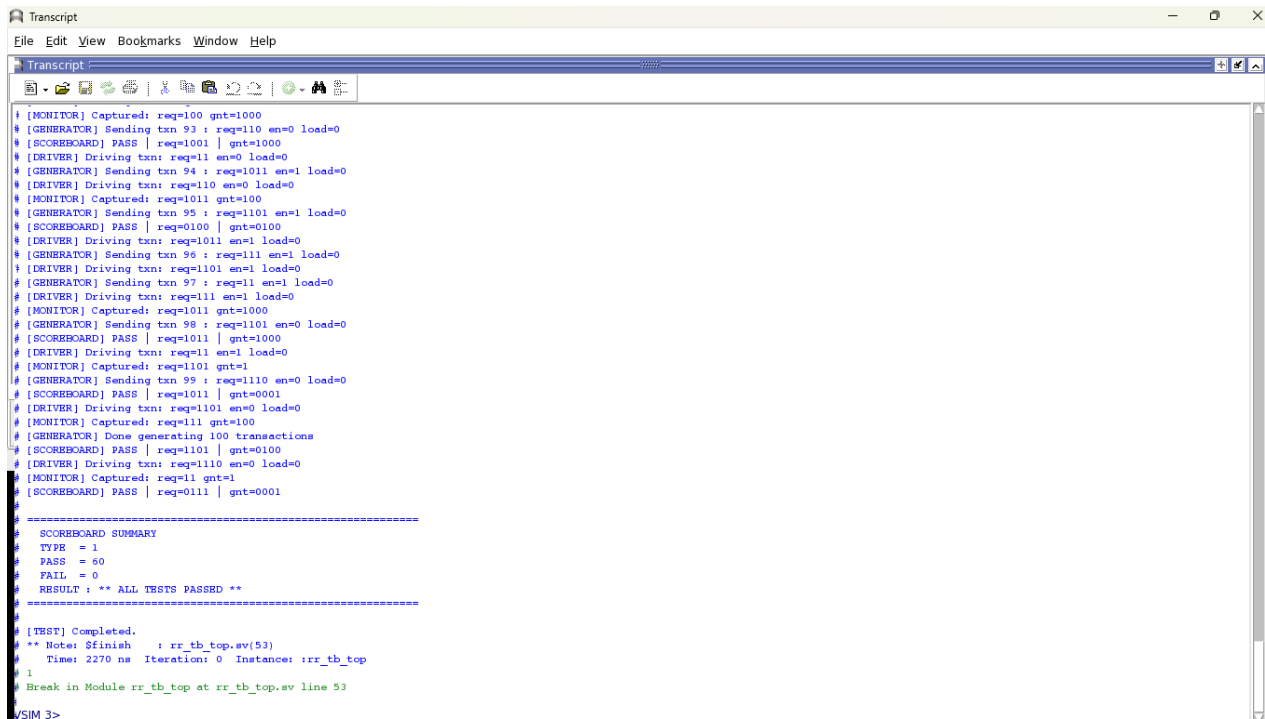


Figure 5.11 — TYPE=0, N=4: Zoomed Waveform.

5.3 TYPE=1 Waveforms (N=4, 5, 7, 10)

TYPE=1 (Modified RR): The pointer jumps to winner+1 after each grant rather than incrementing by 1. This ensures that the next arbitration begins immediately after the last winner, avoiding stale pointer positions.

TYPE=1, N=4



```

Transcript
File Edit View Bookmarks Window Help

# [MONITOR] Captured: req=100 gnt=1000
# [GENERATOR] Sending txn 93 : req=110 en=0 load=0
# [SCOREBOARD] PASS | req=1001 | gnt=1000
# [DRIVER] Driving txn: req=11 en=0 load=0
# [GENERATOR] Sending txn 94 : req=1011 en=1 load=0
# [DRIVER] Driving txn: req=110 en=0 load=0
# [MONITOR] Captured: req=1011 gnt=100
# [GENERATOR] Sending txn 95 : req=1101 en=1 load=0
# [SCOREBOARD] PASS | req=0100 | gnt=0100
# [DRIVER] Driving txn: req=1011 en=1 load=0
# [GENERATOR] Sending txn 96 : req=111 en=1 load=0
# [DRIVER] Driving txn: req=1101 en=1 load=0
# [GENERATOR] Sending txn 97 : req=11 en=1 load=0
# [DRIVER] Driving txn: req=111 en=1 load=0
# [MONITOR] Captured: req=1011 gnt=1000
# [GENERATOR] Sending txn 98 : req=1101 en=0 load=0
# [SCOREBOARD] PASS | req=1011 | gnt=1000
# [DRIVER] Driving txn: req=11 en=1 load=0
# [MONITOR] Captured: req=1101 gnt=1
# [GENERATOR] Sending txn 99 : req=1110 en=0 load=0
# [SCOREBOARD] PASS | req=1011 | gnt=0001
# [DRIVER] Driving txn: req=1101 en=0 load=0
# [MONITOR] Captured: req=111 gnt=100
# [GENERATOR] Done generating 100 transactions
# [SCOREBOARD] PASS | req=1101 | gnt=0100
# [DRIVER] Driving txn: req=1110 en=0 load=0
# [MONITOR] Captured: req=11 gnt=1
# [SCOREBOARD] PASS | req=0111 | gnt=0001

=====
SCOREBOARD SUMMARY
TYPE = 1
PASS = 60
FAIL = 0
RESULT : ** ALL TESTS PASSED **
=====

[Test] Completed.
** Note: $finish : rr_tb_top.sv(53)
Time: 2270 ns Iteration: 0 Instance: :rr_tb_top
1
Break in Module rr_tb_top at rr_tb_top.sv line 53
VSI3M 3>

```

Figure 5.12 — TYPE=1, N=4: Transcript



```

Transcript
File Edit View Bookmarks Window Help

# Loading work.rr_tb_top
# Loading work.round_robin
VSI3M 2> run -all

# [TEST] Starting Round Robin Arbitrator Test (TYPE=1, N=4, W=3)
# [GENERATOR] Sending txn 0 : req=1 en=1 load=0
# [GENERATOR] Sending txn 1 : req=1 en=0 load=0
# [DRIVER] Applying active-low reset
# [DRIVER] Reset released
# [DRIVER] Driving txn: req=1 en=1 load=0
# [GENERATOR] Sending txn 2 : req=101 en=0 load=1
# [DRIVER] Driving txn: req=1 en=0 load=0
# [GENERATOR] Sending txn 3 : req=1100 en=1 load=0
# [DRIVER] Driving txn: req=101 en=0 load=1
# [MONITOR] Captured: req=1 gnt=0
# [GENERATOR] Sending txn 4 : req=1001 en=1 load=0
# [DRIVER] Driving txn: req=1100 en=1 load=0
# [GENERATOR] Sending txn 5 : req=1101 en=1 load=1
# [DRIVER] Driving txn: req=1001 en=1 load=0
# [GENERATOR] Sending txn 6 : req=1000 en=1 load=1
# [DRIVER] Driving txn: req=1101 en=1 load=1
# [MONITOR] Captured: req=1100 gnt=1
# [GENERATOR] Sending txn 7 : req=1101 en=1 load=0
# [SCOREBOARD] PASS | req=0001 | gnt=0001
# [DRIVER] Driving txn: req=1000 en=1 load=1
# [MONITOR] Captured: req=1001 gnt=100
# [GENERATOR] Sending txn 8 : req=1000 en=1 load=1
# [SCOREBOARD] PASS | req=1100 | gnt=0100
# [DRIVER] Driving txn: req=1101 en=1 load=0
# [MONITOR] Captured: req=1101 gnt=1000
# [GENERATOR] Sending txn 9 : req=1001 en=1 load=0
# [SCOREBOARD] PASS | req=1001 | gnt=1000
# [DRIVER] Driving txn: req=1000 en=1 load=1
# [MONITOR] Captured: req=1000 gnt=1
# [GENERATOR] Sending txn 10 : req=11 en=0 load=0
# [SCOREBOARD] PASS | req=1101 | gnt=0001
# [DRIVER] Driving txn: req=1001 en=1 load=0
# [MONITOR] Captured: req=1101 gnt=1000
# [GENERATOR] Sending txn 11 : req=1100 en=0 load=1
# [SCOREBOARD] PASS | req=1000 | gnt=1000
# [DRIVER] Driving txn: req=11 en=0 load=0
# [MONITOR] Captured: req=1000 gnt=1

```

Figure 5.13 — TYPE=1, N=4: Transcript window showing all PASS results.

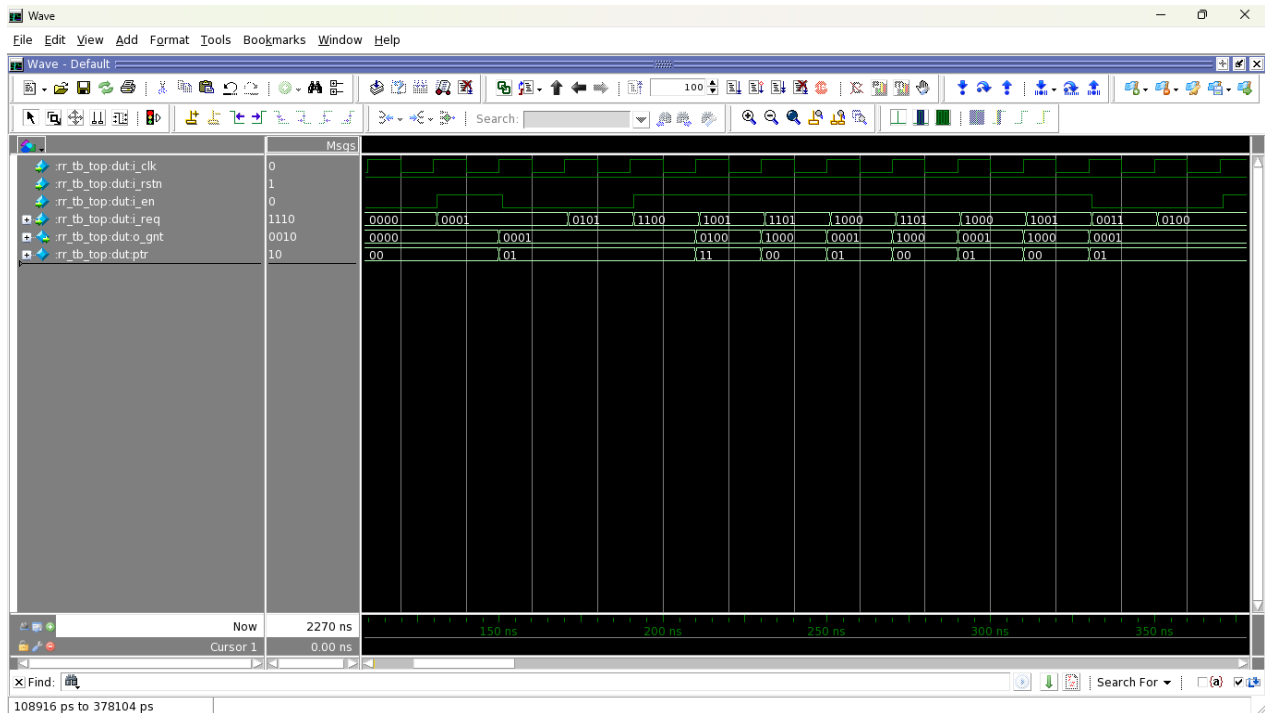


Figure 5.14 — TYPE=1, N=4: Zoomed waveform confirming correct grant-to-pointer alignment.

TYPE=1, N=5

```

Transcript
File Edit View Bookmarks Window Help

# Loading work_rr_if
# Loading work_rr_pkg
# Loading work_rr_tb_top_av_unit
# Loading work_rr_tb_top
# Loading work_round_robin
VSIM 2> run -all

#
# [TEST] Starting Round Robin Arbitrator Test (TYPE=1, N=5, W=3)
#
# [GENERATOR] Sending txn 0 : req=1010 en=1 load=0
# [GENERATOR] Sending txn 1 : req=1 en=0 load=0
# [DRIVER] Applying active-low reset
# [DRIVER] Reset released
# [DRIVER] Driving txn: req=1010 en=1 load=0
# [GENERATOR] Sending txn 2 : req=10000 en=0 load=1
# [DRIVER] Driving txn: req=1 en=0 load=0
# [GENERATOR] Sending txn 3 : req=1011 en=1 load=0
# [DRIVER] Driving txn: req=10000 en=0 load=1
# [MONITOR] Captured: req=1010 gnt=0
# [GENERATOR] Sending txn 4 : req=10001 en=1 load=0
# [DRIVER] Driving txn: req=1011 en=1 load=0
# [GENERATOR] Sending txn 5 : req=1111 en=1 load=1
# [DRIVER] Driving txn: req=10001 en=1 load=0
# [GENERATOR] Sending txn 6 : req=11010 en=1 load=1
# [DRIVER] Driving txn: req=1111 en=1 load=1
# [MONITOR] Captured: req=1011 gnt=10
# [GENERATOR] Sending txn 7 : req=1101 en=1 load=0
# [SCOREBOARD] PASS | req=01010 | gnt=00010
# [DRIVER] Driving txn: req=11010 en=1 load=1
# [MONITOR] Captured: req=10001 gnt=1000
# [GENERATOR] Sending txn 8 : req=111 en=1 load=1
# [SCOREBOARD] PASS | req=01011 | gnt=01000
# [DRIVER] Driving txn: req=1101 en=1 load=0
# [MONITOR] Captured: req=1111 gnt=10000
# [GENERATOR] Sending txn 9 : req=100 en=1 load=0
# [SCOREBOARD] PASS | req=10001 | gnt=10000
# [DRIVER] Driving txn: req=111 en=1 load=1
# [MONITOR] Captured: req=11010 gnt=1
# [GENERATOR] Sending txn 10 : req=11011 en=0 load=0
# [SCOREBOARD] PASS | req=01111 | gnt=00001
# [DRIVER] Driving txn: req=100 en=1 load=0
# [MONITOR] Captured: req=1101 gnt=10

```

Figure 5.15 — TYPE=1, N=5: Transcript.

```

# [GENERATOR] Sending txn 93 : req=11111 en=0 load=0
# [SCOREBOARD] PASS | req=00010 | gnt=00010
# [DRIVER] Driving txn: req=11110 en=0 load=0
# [GENERATOR] Sending txn 94 : req=111 en=1 load=0
# [DRIVER] Driving txn: req=11111 en=0 load=0
# [MONITOR] Captured: req=11111 gnt=1000
# [GENERATOR] Sending txn 95 : req=10111 en=1 load=0
# [SCOREBOARD] PASS | req=01001 | gnt=01000
# [DRIVER] Driving txn: req=111 en=1 load=0
# [GENERATOR] Sending txn 96 : req=1 en=1 load=0
# [DRIVER] Driving txn: req=10111 en=1 load=0
# [GENERATOR] Sending txn 97 : req=101 en=1 load=0
# [DRIVER] Driving txn: req=1 en=1 load=0
# [MONITOR] Captured: req=111 gnt=10000
# [GENERATOR] Sending txn 98 : req=1110 en=0 load=0
# [SCOREBOARD] PASS | req=11111 | gnt=10000
# [DRIVER] Driving txn: req=101 en=1 load=0
# [MONITOR] Captured: req=10111 gnt=1
# [GENERATOR] Sending txn 99 : req=1 en=0 load=0
# [SCOREBOARD] PASS | req=00111 | gnt=00001
# [DRIVER] Driving txn: req=1110 en=0 load=0
# [MONITOR] Captured: req=11 gnt=10
# [GENERATOR] Done generating 100 transactions
# [SCOREBOARD] PASS | req=10111 | gnt=00010
# [DRIVER] Driving txn: req=1 en=0 load=0
# [MONITOR] Captured: req=101 gnt=1
# [SCOREBOARD] PASS | req=00011 | gnt=00001
#
=====
# SCOREBOARD SUMMARY
# TYPE = 1
# PASS = 60
# FAIL = 0
# RESULT : ** ALL TESTS PASSED **
=====
# [TEST] Completed.
# ** Note: $finish : rr_tb_top.sv(53)
# Time: 2270 ns Iteration: 0 Instance: rrr_tb_top
# 1
# Break in Module rr_tb_top at rr_tb_top.sv line 53
VSIM 3>

```

Figure 5.16 — TYPE=1, N=5: Transcript.

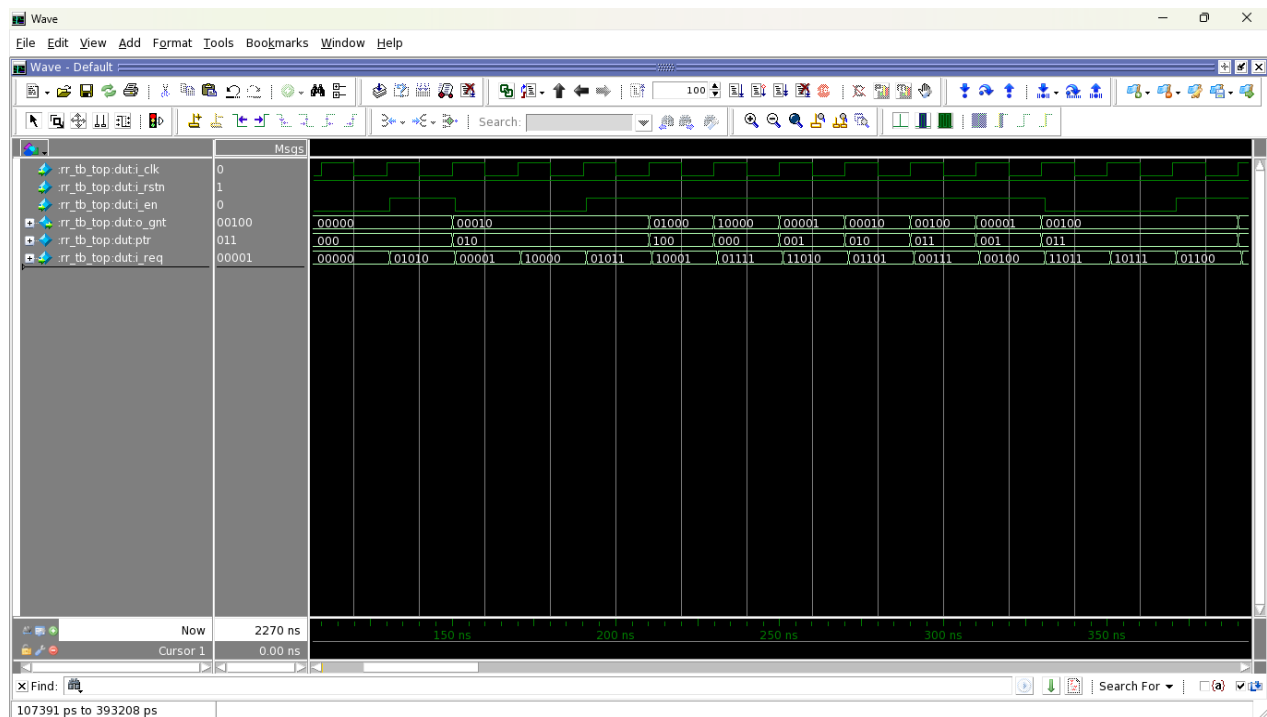


Figure 5.17 — TYPE=1, N=5: Zoomed Waveform.

TYPE=1, N=7

```

Transcript
File Edit View Bookmarks Window Help

# ** Warning: (VSI08-8891) All optimizations are turned off because the -novopt switch is in effect. This will cause your simulation to run very slowly. If you are using this switch to preserve visibility for Debug or PLI features please see the User's Manual section on Preserving Object Visibility with vopt.
# Loading sv_std.std
# Loading work_rr_if
# Loading work_rr_pkg
# Loading work_rr_tb_top_sv_unit
# Loading work_rr_tb_top
# Loading wrk_round_robin

VSI02 2> run -all
#
# [TEST] Starting Round Robin Arbitrator Test (TYPE=1, N=7, W=3)
#
# [GENERATOR] Sending txn 0 : req=1111000 en=1 load=0
# [GENERATOR] Sending txn 1 : req=1110011 en=0 load=0
# [DRIVER] Applying active-low reset
# [DRIVER] Reset released
# [DRIVER] Driving txn: req=1111000 en=1 load=0
# [GENERATOR] Sending txn 2 : req=1110110 en=0 load=1
# [DRIVER] Driving txn: req=1110011 en=0 load=0
# [GENERATOR] Sending txn 3 : req=10001 en=1 load=0
# [DRIVER] Driving txn: req=1110110 en=0 load=1
# [MONITOR] Captured: req=1111000 gnt=0
# [GENERATOR] Sending txn 4 : req=1111110 en=1 load=0
# [DRIVER] Driving txn: req=10001 en=1 load=0
# [GENERATOR] Sending txn 5 : req=1001011 en=1 load=1
# [DRIVER] Driving txn: req=1111110 en=1 load=0
# [GENERATOR] Sending txn 6 : req=1011011 en=1 load=1
# [DRIVER] Driving txn: req=1001011 en=1 load=1
# [MONITOR] Captured: req=10001 gnt=1000
# [GENERATOR] Sending txn 7 : req=1111111 en=1 load=0
# [SCOREBOARD] PASS | req=1111000 | gnt=0001000
# [DRIVER] Driving txn: req=1011011 en=1 load=1
# [MONITOR] Captured: req=1111110 gnt=10000
# [GENERATOR] Sending txn 8 : req=1111111 en=1 load=1
# [SCOREBOARD] PASS | req=0010001 | gnt=0010000
# [DRIVER] Driving txn: req=1111111 en=1 load=0
# [MONITOR] Captured: req=1001011 gnt=100000
# [GENERATOR] Sending txn 9 : req=1110111 en=1 load=0
# [SCOREBOARD] PASS | req=1111110 | gnt=0100000
# [DRIVER] Driving txn: req=1111111 en=1 load=1
# [MONITOR] Captured: req=1011011 gnt=1000000
# [GENERATOR] Sending txn 10 : req=111 en=0 load=0
# [SCOREBOARD] PASS | req=1001011 | gnt=1000000

```

Figure 5.18 — TYPE=1, N=7: Transcript.

```

Transcript
File Edit View Bookmarks Window Help

# [GENERATOR] Sending txn 93 : req=110011 en=0 load=0
# [SCOREBOARD] PASS | req=1000001 | gnt=1000000
# [DRIVER] Driving txn: req=1111000 en=0 load=0
# [GENERATOR] Sending txn 94 : req=1001000 en=1 load=0
# [DRIVER] Driving txn: req=110011 en=1 load=0
# [MONITOR] Captured: req=1101 gnt=10000
# [GENERATOR] Sending txn 95 : req=110000 en=1 load=0
# [SCOREBOARD] PASS | req=1010000 | gnt=0010000
# [DRIVER] Driving txn: req=1001000 en=1 load=0
# [GENERATOR] Sending txn 96 : req=1010 en=1 load=0
# [DRIVER] Driving txn: req=110000 en=1 load=0
# [GENERATOR] Sending txn 97 : req=1111 en=1 load=0
# [DRIVER] Driving txn: req=1010 en=1 load=0
# [MONITOR] Captured: req=1001000 gnt=1
# [GENERATOR] Sending txn 98 : req=1000 en=0 load=0
# [SCOREBOARD] PASS | req=0001101 | gnt=00000001
# [DRIVER] Driving txn: req=1111 en=1 load=0
# [MONITOR] Captured: req=110000 gnt=1000
# [GENERATOR] Sending txn 99 : req=1101011 en=0 load=0
# [SCOREBOARD] PASS | req=1001000 | gnt=0001000
# [DRIVER] Driving txn: req=1000 en=0 load=0
# [MONITOR] Captured: req=1010 gnt=10000
# [GENERATOR] Done generating 100 transactions
# [SCOREBOARD] PASS | req=0110000 | gnt=0010000
# [DRIVER] Driving txn: req=1101011 en=0 load=0
# [MONITOR] Captured: req=1111 gnt=10
# [SCOREBOARD] PASS | req=0001010 | gnt=00000010
#
# =====
# SCOREBOARD SUMMARY
# TYPE = 1
# PASS = 60
# FAIL = 0
# RESULT : ** ALL TESTS PASSED **
# =====
#
# [TEST] Completed.
# ** Note: $finish : rr_tb_top.sv(53)
# Time: 2270 ns Iteration: 0 Instance: rr_tb_top
# 1
# Break in Module rr_tb_top at rr_tb_top.sv line 53
VSI03 3>

```

Figure 5.19 — TYPE=1, N=7: Transcript.

TYPE=1, N=10

```

Transcript
File Edit View Bookmarks Window Help

//
QuestaSim> vsim work.rr_tb_top
# vsim work.rr_tb_top
# Start time: 16:20:45 on Apr 12, 2026
# ** Warning: (vsim-8891) All optimizations are turned off because the -novopt switch is in effect. This will cause your simulation to run very slowly. If you are using this switch to preserve
# visibility for Debug or PLI features please see the User's Manual section on Preserving Object Visibility with vopt.
# Loading sv_std.std
# Loading work.rr_if
# Loading work.rr_pkg
# Loading work.rr_tb_top_sv_unit
# Loading work.rr_tb_top
# Loading work.round_robin
VSIM 2> run -all
#
# [TEST] Starting Round Robin Arbiter Test (TYPE=1, N=10, W=3)
#
# [GENERATOR] Sending txn 0 : req=1000100001 en=1 load=0
# [GENERATOR] Sending txn 1 : req=101000 en=0 load=0
# [DRIVER] Applying active-low reset
# [DRIVER] Reset released
# [DRIVER] Driving txn: req=1000100001 en=1 load=0
# [GENERATOR] Sending txn 2 : req=11111111 en=0 load=1
# [DRIVER] Driving txn: req=101000 en=0 load=0
# [GENERATOR] Sending txn 3 : req=110001100 en=1 load=0
# [DRIVER] Driving txn: req=11111111 en=0 load=1
# [MONITOR] Captured: req=1000100001 gnt=0
# [GENERATOR] Sending txn 4 : req=11101 en=1 load=0
# [DRIVER] Driving txn: req=110001100 en=1 load=0
# [GENERATOR] Sending txn 5 : req=1000110010 en=1 load=1
# [DRIVER] Driving txn: req=11101 en=1 load=0
# [GENERATOR] Sending txn 6 : req=10001001 en=1 load=1
# [DRIVER] Driving txn: req=1000110010 en=1 load=1
# [MONITOR] Captured: req=10001100 gnt=1
# [GENERATOR] Sending txn 7 : req=1110001111 en=1 load=0
# [SCOREBOARD] PASS | req=1000100001 | gnt=0000000001
# [DRIVER] Driving txn: req=10001001 en=1 load=1
# [MONITOR] Captured: req=11101 gnt=100
# [GENERATOR] Sending txn 8 : req=1010001010 en=1 load=1
# [SCOREBOARD] PASS | req=0110001100 | gnt=0000000100
# [DRIVER] Driving txn: req=1110001111 en=1 load=0
# [MONITOR] Captured: req=1000110010 gnt=1000
# [GENERATOR] Sending txn 9 : req=1110111111 en=1 load=0
# [SCOREBOARD] PASS | req=0000011101 | gnt=0000001000

```

Figure 5.20 — TYPE=1, N=10: Transcript.

```

# [GENERATOR] Sending txn 93 : req=1110111101 en=0 load=0
# [SCOREBOARD] PASS | req=100011101 | gnt=1000000000
# [DRIVER] Driving txn: req=1100110100 en=0 load=0
# [GENERATOR] Sending txn 94 : req=10001110 en=1 load=0
# [DRIVER] Driving txn: req=1101111101 en=0 load=0
# [MONITOR] Captured: req=100101 gnt=1
# [GENERATOR] Sending txn 95 : req=110101 en=1 load=0
# [SCOREBOARD] PASS | req=0010100001 | gnt=0000000001
# [DRIVER] Driving txn: req=110001110 en=1 load=0
# [GENERATOR] Sending txn 96 : req=1110010 en=1 load=0
# [DRIVER] Driving txn: req=110101 en=1 load=0
# [GENERATOR] Sending txn 97 : req=1001111011 en=1 load=0
# [DRIVER] Driving txn: req=1110010 en=1 load=0
# [MONITOR] Captured: req=110001110 gnt=100
# [GENERATOR] Sending txn 98 : req=1100110010 en=0 load=0
# [SCOREBOARD] PASS | req=0000100101 | gnt=0000000100
# [DRIVER] Driving txn: req=1001111011 en=1 load=0
# [MONITOR] Captured: req=110101 gnt=1000
# [GENERATOR] Sending txn 99 : req=1000110110 en=0 load=0
# [SCOREBOARD] PASS | req=0110001110 | gnt=0000000100
# [DRIVER] Driving txn: req=1100110010 en=0 load=0
# [MONITOR] Captured: req=1110010 gnt=10000
# [GENERATOR] Done generating 100 transactions
# [SCOREBOARD] PASS | req=0000110101 | gnt=0000010000
# [DRIVER] Driving txn: req=1000110110 en=0 load=0
# [MONITOR] Captured: req=1001111011 gnt=100000
# [SCOREBOARD] PASS | req=0001110010 | gnt=0000100000
#
# =====
# SCOREBOARD SUMMARY
# TYPE = 1
# PASS = 60
# FAIL = 0
# RESULT : ** ALL TESTS PASSED **
# =====
# [TEST] Completed.
# ** Note: $finish : rr_tb_top.sv(53)
# Time: 2270 ns Iteration: 0 Instance: rr_tb_top
# 1
# Break in Module rr_tb_top at rr_tb_top.sv line 53
VSIM 3>

```

Figure 5.21 — TYPE=1, N=10: Transcript.

5.4 TYPE=2 Waveforms (N=4, 5, 10)

TYPE=2 (Weighted RR): Weight counters are visible on the waveform. The `i_weights` input is loaded via `i_load`, after which `weight_counters` reflects the new values. On each grant, the winner's weight counter decrements. When a counter reaches 0, that requester drops out of the highest-priority tier until weights are reloaded.

TYPE=2, N=4



```

Transcript
File Edit View Bookmarks Window Help

QuestaSim> vsim work.rr_tb_top
# vsim work.rr_tb_top
# Start time: 18:04:58 on Apr 12, 2026
# ** Warning: (vsim-8891) All optimizations are turned off because the -novopt switch is in effect. This will cause your simulation to run very slowly. If you are using this switch to preserve visibility for Debug or PLI features please see the User's Manual section on Preserving Object Visibility with vopt.
# Loading sv_std.std
# Loading work.rr_if
# Loading work.rr_pkg
# Loading work.rr_tb_top_sv_unit
# Loading work.rr_tb_top
# Loading work.round_robin
VSIM 2> run -all
#
# [TEST] Starting Round Robin Arbitrator Test (TYPE=2, N=4, W=3)
#
# [GENERATOR] Sending txn 0 : req=1 en=1 load=0
# [GENERATOR] Sending txn 1 : req=1 en=0 load=0
# [DRIVER] Applying active-low reset
# [DRIVER] Reset released
# [DRIVER] Driving txn: req=1 en=1 load=0
# [GENERATOR] Sending txn 2 : req=101 en=0 load=1
# [DRIVER] Driving txn: req=1 en=0 load=0
# [GENERATOR] Sending txn 3 : req=100 en=1 load=0
# [DRIVER] Driving txn: req=101 en=0 load=1
# [MONITOR] Captured: req=1 gnt=0
# [GENERATOR] Sending txn 4 : req=1001 en=1 load=0
# [DRIVER] Driving txn: req=100 en=1 load=0
# [GENERATOR] Sending txn 5 : req=101 en=1 load=1
# [DRIVER] Driving txn: req=1001 en=1 load=0
# [GENERATOR] Sending txn 6 : req=1000 en=1 load=1
# [DRIVER] Driving txn: req=1101 en=1 load=1
# [MONITOR] Captured: req=1100 gnt=1
# [GENERATOR] Sending txn 7 : req=1101 en=1 load=0
# [SCOREBOARD] PASS | req=0001 | gnt=0001
# [DRIVER] Driving txn: req=1000 en=1 load=1
# [MONITOR] Captured: req=1001 gnt=100
# [GENERATOR] Sending txn 8 : req=1000 en=1 load=1
# [SCOREBOARD] PASS | req=1100 | gnt=0100
# [DRIVER] Driving txn: req=1101 en=1 load=0
# [MONITOR] Captured: req=1101 gnt=1
# [GENERATOR] Sending txn 9 : req=1001 en=1 load=0
# [SCOREBOARD] PASS | req=1001 | gnt=0001
# [DRIVER] Driving txn: req=1000 en=1 load=1

```

Figure 5.22 — TYPE=2, N=4: Transcript.

```

# [SCOREBOARD] PASS | req=1001 | gnt=0001
# [DRIVER] Driving txn: req=11 en=0 load=0
# [GENERATOR] Sending txn 94 : req=1011 en=1 load=0
# [DRIVER] Driving txn: req=110 en=0 load=0
# [MONITOR] Captured: req=1011 gnt=100
# [GENERATOR] Sending txn 95 : req=1101 en=1 load=0
# [SCOREBOARD] PASS | req=0100 | gnt=0100
# [DRIVER] Driving txn: req=1011 en=1 load=0
# [GENERATOR] Sending txn 96 : req=111 en=1 load=0
# [DRIVER] Driving txn: req=1101 en=1 load=0
# [GENERATOR] Sending txn 97 : req=11 en=1 load=0
# [DRIVER] Driving txn: req=111 en=1 load=0
# [MONITOR] Captured: req=1011 gnt=1
# [GENERATOR] Sending txn 98 : req=1101 en=0 load=0
# [SCOREBOARD] PASS | req=1011 | gnt=0001
# [DRIVER] Driving txn: req=11 en=1 load=0
# [MONITOR] Captured: req=1101 gnt=1
# [GENERATOR] Sending txn 99 : req=1110 en=0 load=0
** Error: [SCOREBOARD] FAIL | req=1011 | gnt=0001 | exp=0010 | ptr=1
# Time: 2050 ns Scope: rr_pkg.rr_scoreboard.rr_scoreboard_1.run File: rr_scoreboard.sv Line: 135
# [DRIVER] Driving txn: req=1101 en=0 load=0
# [MONITOR] Captured: req=111 gnt=100
# [GENERATOR] Done generating 100 transactions
# [SCOREBOARD] PASS | req=1101 | gnt=0100
# [DRIVER] Driving txn: req=1110 en=0 load=0
# [MONITOR] Captured: req=11 gnt=100
# [SCOREBOARD] PASS | req=0111 | gnt=0100
#
# =====
# SCOREBOARD SUMMARY
# TYPE = 2
# PASS = 41
# FAIL = 19
# RESULT : !! 19 TESTS FAILED !!
# =====
# [TEST] Completed.
** Note: $finish : rr_tb_top.sv(53)
# Time: 2270 ns Iteration: 0 Instance: rrr_tb_top
# 1
# Break in Module rr_tb_top at rr_tb_top.sv line 53
VSIM 3>

```

Figure 5.23 — TYPE=2, N=4: Transcript

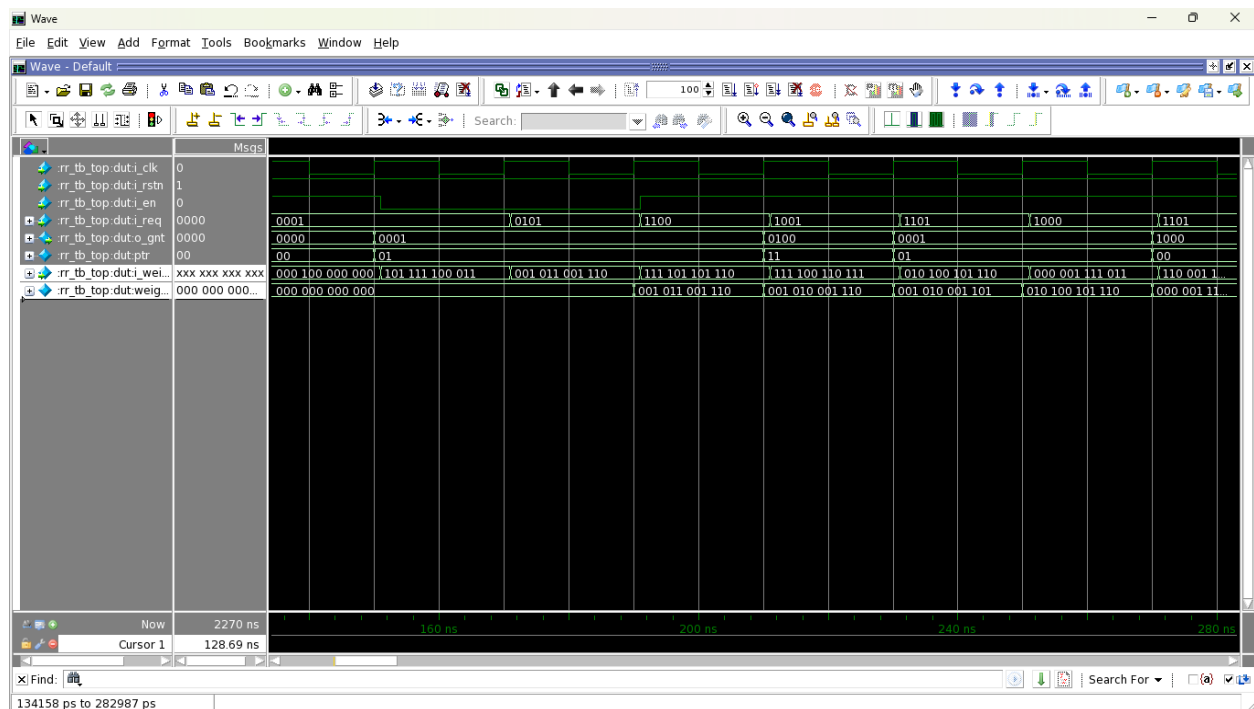


Figure 5.24 — TYPE=2, N=4: Waveform showing weighted_counters loading and decrementing.

TYPE=2, N=5



```

Transcript
File Edit View Bookmarks Window Help

# veim work.rr.tb_top
# Start time: 19:10:45 on Apr 12, 2026
# ** Warning: (vsim-8891) All optimisations are turned off because the -novopt switch is in effect. This will cause your simulation to run very slowly. If you are using this switch to preserv
e visibility for Debug or PLI features please see the User's Manual section on Preserving Object Visibility with vopt.
# Loading sv_std.std
# Loading work.rr_if
# Loading work.rr_pkg
# Loading work.rr.tb_top_sv_unit
# Loading work.rr.tb_top
# Loading work.round_robin
VSIM 2> run -all
#
# [TEST] Starting Round Robin Arbitrator Test (TYPE=2, N=5, W=3)
#
# [GENERATOR] Sending txn 0 : req=1010 en=1 load=0
# [GENERATOR] Sending txn 1 : req=1 en=0 load=0
# [DRIVER] Applying active-low reset
# [DRIVER] Reset released
# [DRIVER] Driving txn: req=1010 en=1 load=0
# [GENERATOR] Sending txn 2 : req=10000 en=0 load=1
# [DRIVER] Driving txn: req=1 en=0 load=0
# [GENERATOR] Sending txn 3 : req=1011 en=1 load=0
# [DRIVER] Driving txn: req=10000 en=0 load=1
# [MONITOR] Captured: req=1010 gnt=0
# [GENERATOR] Sending txn 4 : req=10001 en=1 load=0
# [DRIVER] Driving txn: req=1011 en=1 load=0
# [GENERATOR] Sending txn 5 : req=1111 en=1 load=1
# [DRIVER] Driving txn: req=10001 en=1 load=0
# [GENERATOR] Sending txn 6 : req=11010 en=1 load=1
# [DRIVER] Driving txn: req=1111 en=1 load=1
# [MONITOR] Captured: req=1011 gnt=10
# [GENERATOR] Sending txn 7 : req=1101 en=1 load=0
# [SCOREBOARD] PASS | req=01010 | gnt=00010
# [DRIVER] Driving txn: req=11010 en=1 load=1
# [MONITOR] Captured: req=10001 gnt=1000
# [GENERATOR] Sending txn 8 : req=111 en=1 load=1
# [SCOREBOARD] PASS | req=01011 | gnt=01000
# [DRIVER] Driving txn: req=1101 en=1 load=0
# [MONITOR] Captured: req=1111 gnt=10000
# [GENERATOR] Sending txn 9 : req=100 en=1 load=0
# [SCOREBOARD] PASS | req=10001 | gnt=10000
# [DRIVER] Driving txn: req=111 en=1 load=1
# [MONITOR] Captured: req=11010 en=1 load=1

```

Figure 5.25 — TYPE=2, N=5: Transcript.



```

Transcript
File Edit View Bookmarks Window Help

# [DRIVER] Driving txn: req=1110 en=0 load=0
# [GENERATOR] Sending txn 94 : req=111 en=1 load=0
# [DRIVER] Driving txn: req=11111 en=0 load=0
# [MONITOR] Captured: req=11111 gnt=1000
# [GENERATOR] Sending txn 95 : req=10111 en=1 load=0
# [SCOREBOARD] PASS | req=01001 | gnt=01000
# [DRIVER] Driving txn: req=111 en=1 load=0
# [GENERATOR] Sending txn 96 : req=11 en=1 load=0
# [DRIVER] Driving txn: req=10111 en=1 load=0
# [GENERATOR] Sending txn 97 : req=101 en=1 load=0
# [DRIVER] Driving txn: req=11 en=1 load=0
# [MONITOR] Captured: req=111 gnt=1000
# [GENERATOR] Sending txn 98 : req=1110 en=0 load=0
# [SCOREBOARD] PASS | req=11111 | gnt=01000
# [DRIVER] Driving txn: req=101 en=1 load=0
# [MONITOR] Captured: req=10111 gnt=1
# [GENERATOR] Sending txn 99 : req=1 en=0 load=0
# [SCOREBOARD] PASS | req=00111 | gnt=00001
# [DRIVER] Driving txn: req=1110 en=0 load=0
# [MONITOR] Captured: req=11 gnt=10
# [GENERATOR] Done generating 100 transactions
# ** Error: [SCOREBOARD] FAIL | req=10111 | gnt=00010 | exp=10000 | ptr=1
# Time: 2070 ns Scope: rr_pkg.rr_scoreboard.rr_scoreboard_1.run File: rr_scoreboard.sv Line: 135
# [DRIVER] Driving txn: req=1 en=0 load=0
# [MONITOR] Captured: req=101 gnt=1
# ** Error: [SCOREBOARD] FAIL | req=00011 | gnt=00001 | exp=00010 | ptr=1
# Time: 2090 ns Scope: rr_pkg.rr_scoreboard.rr_scoreboard_1.run File: rr_scoreboard.sv Line: 135
# =====
# SCOREBOARD SUMMARY
# TYPE = 2
# PASS = 40
# FAIL = 20
# RESULT : !! 20 TESTS FAILED !!
# =====
# [TEST] Completed.
# ** Note: $finish : rr_tb_top.sv(53)
# Time: 2270 ns Iteration: 0 Instance: rr_tb_top
# 1
# Break in Module rr_tb_top at rr_tb_top.sv line 53
VSIM 3>

```

Figure 5.26 — TYPE=2, N=5: Transcript.

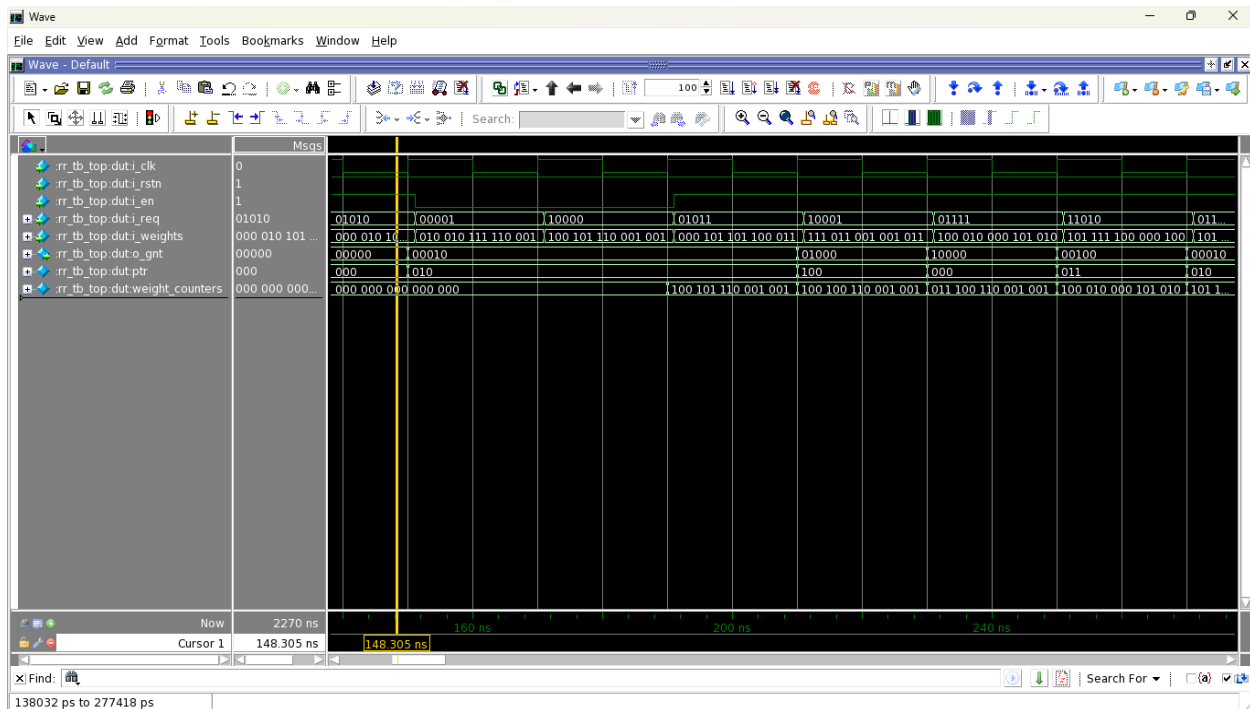


Figure 5.27 — TYPE=2, N=5: Waveform.

TYPE=2, N=10

```

Transcript
File Edit View Bookmarks Window Help

# Loading sv_std.v
# Loading work_rr_if
# Loading work_rr_pkg
# Loading work_rr_tb_top_sv_unit
# Loading work_rr_tb_top
# Loading work_round_robin
V$IM 2> run -all
#
# [TEST] Starting Round Robin Arbiter Test (TYPE=2, N=10, W=3)
#
# [GENERATOR] Sending txn 0 : req=1000100001 en=1 load=0
# [GENERATOR] Sending txn 1 : req=101000 en=0 load=0
# [DRIVER] Applying active-low reset
# [DRIVER] Reset released
# [DRIVER] Driving txn: req=1000100001 en=1 load=0
# [GENERATOR] Sending txn 2 : req=11111111 en=0 load=1
# [DRIVER] Driving txn: req=101000 en=0 load=0
# [GENERATOR] Sending txn 3 : req=110001100 en=1 load=0
# [DRIVER] Driving txn: req=11111111 en=0 load=1
# [MONITOR] Captured: req=1000100001 gnt=0
# [GENERATOR] Sending txn 4 : req=11101 en=1 load=0
# [DRIVER] Driving txn: req=110001100 en=1 load=0
# [GENERATOR] Sending txn 5 : req=1000110010 en=1 load=1
# [DRIVER] Driving txn: req=11101 en=1 load=0
# [GENERATOR] Sending txn 6 : req=10001001 en=1 load=1
# [DRIVER] Driving txn: req=1000110010 en=1 load=1
# [MONITOR] Captured: req=110001100 gnt=1
# [GENERATOR] Sending txn 7 : req=1110001111 en=1 load=0
# [SCOREBOARD] PASS | req=1000100001 | gnt=0000000001
# [DRIVER] Driving txn: req=10001001 en=1 load=1
# [MONITOR] Captured: req=11101 gnt=1000
# [GENERATOR] Sending txn 8 : req=1010001010 en=1 load=1
# ** Error: [SCOREBOARD] FAIL | req=0110001100 | exp=0000000100 | ptr=1
# Time: 230 ns Scope: rr_pkg.rr_scoreboard.rr_scoreboard_1.run File: rr_scoreboard.v Line: 135
# [DRIVER] Driving txn: req=1110001111 en=1 load=0
# [MONITOR] Captured: req=1000110010 gnt=10000
# [GENERATOR] Sending txn 9 : req=1110111111 en=1 load=0
# [SCOREBOARD] PASS | req=0000011101 | gnt=0000010000
# [DRIVER] Driving txn: req=1010001010 en=1 load=1
# [MONITOR] Captured: req=10001001 gnt=10000
# [GENERATOR] Sending txn 10 : req=10100110 en=0 load=0
# [SCOREBOARD] PASS | req=1000110010 | gnt=0000010000

```

Figure 5.28 — TYPE=2, N=10: Transcript.

```

# [SCOREBOARD] PASS | req=1000011101 | gnt=0000001000
# [DRIVER] Driving txn: req=1100110100 en=0 load=0
# [GENERATOR] Sending txn 94 : req=110001110 en=1 load=0
# [DRIVER] Driving txn: req=110111101 en=0 load=0
# [MONITOR] Captured: req=100101 gnt=10
# [GENERATOR] Sending txn 95 : req=110101 en=1 load=0
# ** Error: [SCOREBOARD] FAIL | req=0010100011 | gnt=0000000010 | exp=0000100000 | ptr=4
# Time: 1970 ns Scope: rr_pkg.rr_scoreboard.rr_scoreboard__1.run File: rr_scoreboard.sv Line: 135
# [DRIVER] Driving txn: req=110001110 en=1 load=0
# [GENERATOR] Sending txn 96 : req=1110010 en=1 load=0
# [DRIVER] Driving txn: req=110101 en=1 load=0
# [GENERATOR] Sending txn 97 : req=100111101 en=1 load=0
# [DRIVER] Driving txn: req=1110010 en=1 load=0
# [MONITOR] Captured: req=110001110 gnt=100000
# [GENERATOR] Sending txn 98 : req=1100110010 en=0 load=0
# [SCOREBOARD] PASS | req=0000100101 | gnt=0000100000
# [DRIVER] Driving txn: req=1001111011 en=1 load=0
# [MONITOR] Captured: req=110101 gnt=10
# [GENERATOR] Sending txn 99 : req=1000110110 en=0 load=0
# [SCOREBOARD] PASS | req=0110001110 | gnt=0000000010
# [DRIVER] Driving txn: req=1100110010 en=0 load=0
# [MONITOR] Captured: req=1110010 gnt=1
# [GENERATOR] Done generating 100 transactions
# [SCOREBOARD] PASS | req=0000110101 | gnt=0000000001
# [DRIVER] Driving txn: req=1000110110 en=0 load=0
# [MONITOR] Captured: req=1001111011 gnt=10
# [SCOREBOARD] PASS | req=0001110010 | gnt=0000000010
#
# =====
# SCOREBOARD SUMMARY
# TYPE = 2
# PASS = 37
# FAIL = 23
# RESULT : !! 23 TESTS FAILED !!
# =====
# [TEST] Completed.
# ** Note: $finish : rr_tb_top.sv(53)
# Time: 2270 ns Iteration: 0 Instance: rrr_tb_top
# 1
# Break in Module rr_tb_top at rr_tb_top.sv line 53
VSIM 3>

```

Figure 5.29 — TYPE=2, N=10: Transcript.

6. Code Coverage Analysis

6.1 Methodology

Code coverage was collected using Questa's built-in coverage instrumentation. The `+cover=bcesf` flag enables statement (s), branch (b), condition (c), expression (e), and FSM (f) coverage. Coverage data from all 12 simulation runs (TYPE=0,1,2 x N=4,5,7,10) was merged into a single database using `vcover merge` and reported with `vcover report`.

```
# Compile with coverage
vlog +define+N=7+W=3+TYPE=0 +cover=bcesf round_robin.sv rr_interface.sv rr_pkg.sv rr_tb_top.sv

# Simulate and save coverage
vsim -c -novopt -coverage \
-do "coverage save -onexit cov_TYPE0_N7.ucdb; run -all; quit" rr_tb_top

# Merge all runs
vcover merge combined.ucdb cov_TYPE*.ucdb

# Report
vcover report combined.ucdb
```

6.2 Coverage Results

The coverage report for the combined runs is shown below. The DUT (`round_robin.sv`) achieved near-complete coverage across all metrics.

File: round_robin.sv				
Enabled Coverage	Active	Hits	Misses	% Covered
Stmts	33	33	0	100.0
Branches	17	17	0	100.0
FEC Condition Terms	5	4	1	80.0
FEC Expression Terms	0	0	0	100.0
FSMs				100.0
States	0	0	0	100.0
Transitions	0	0	0	100.0
File: rr_driver.sv				
Enabled Coverage	Active	Hits	Misses	% Covered
Stmts	21	21	0	100.0
Branches	0	0	0	100.0
FEC Condition Terms	0	0	0	100.0
FEC Expression Terms	0	0	0	100.0
FSMs				100.0
States	0	0	0	100.0
Transitions	0	0	0	100.0
File: rr_environment.sv				
Enabled Coverage	Active	Hits	Misses	% Covered
Stmts	16	16	0	100.0
Branches	0	0	0	100.0
FEC Condition Terms	0	0	0	100.0
FEC Expression Terms	0	0	0	100.0
FSMs				100.0
States	0	0	0	100.0
Transitions	0	0	0	100.0

Figure 6.1 — Questa vcover report: per-file coverage summary.

```

=====
=== File: rr_generator.sv
=====
Enabled Coverage      Active      Hits      Misses % Covered
-----
Stmts                13           9         4     69.2
Branches             4           2         2     50.0
FEC Condition Terms  0           0         0    100.0
FEC Expression Terms 0           0         0    100.0
FSMs                 0           0         0    100.0
  States             0           0         0    100.0
  Transitions        0           0         0    100.0
=====

=== File: rr_interface.sv
=====
Enabled Coverage      Active      Hits      Misses % Covered
-----
Stmts                 1           1         0    100.0
Branches             0           0         0    100.0
FEC Condition Terms  0           0         0    100.0
FEC Expression Terms 0           0         0    100.0
FSMs                 0           0         0    100.0
  States             0           0         0    100.0
  Transitions        0           0         0    100.0
=====

=== File: rr_monitor.sv
=====
Enabled Coverage      Active      Hits      Misses % Covered
-----
Stmts                12          12         0    100.0
Branches             2           2         0    100.0
FEC Condition Terms  0           0         0    100.0
FEC Expression Terms 0           0         0    100.0
FSMs                 0           0         0    100.0
  States             0           0         0    100.0
  Transitions        0           0         0    100.0
=====

```

Figure 6.2 — Questa vcover report: per-file coverage summary.

```

=====
=== File: rr_scoreboard.sv
=====
Enabled Coverage      Active      Hits      Misses % Covered
-----
Stmts                60          44         16     73.3
Branches            33          20         13     60.6
FEC Condition Terms  7           2         5     28.5
FEC Expression Terms 0           0         0    100.0
FSMs                 0           0         0    100.0
  States             0           0         0    100.0
  Transitions        0           0         0    100.0
=====

=== File: rr_tb_top.sv
=====
Enabled Coverage      Active      Hits      Misses % Covered
-----
Stmts                 8           7         1     87.5
Branches             0           0         0    100.0
FEC Condition Terms  0           0         0    100.0
FEC Expression Terms 0           0         0    100.0
FSMs                 0           0         0    100.0
  States             0           0         0    100.0
  Transitions        0           0         0    100.0
=====

=== File: rr_test.sv
=====
Enabled Coverage      Active      Hits      Misses % Covered
-----
Stmts                 5           5         0    100.0
Branches             0           0         0    100.0
FEC Condition Terms  0           0         0    100.0
FEC Expression Terms 0           0         0    100.0
FSMs                 0           0         0    100.0
  States             0           0         0    100.0
  Transitions        0           0         0    100.0
=====

```

Figure 6.3 — Questa vcover report: per-file coverage summary.

```

=====
=== File: rr_transaction.sv
=====
Enabled Coverage      Active      Hits      Misses % Covered
-----
Stmts                  8           0           8       0.0
Branches               0           0           0     100.0
FEC Condition Terms    0           0           0     100.0
FEC Expression Terms   0           0           0     100.0
FSMs                   0           0           0     100.0
  States               0           0           0     100.0
  Transitions          0           0           0     100.0

```

Figure 6.4 — Questa vcover report: per-file coverage summary.

6.3 Coverage Summary Table

File	Stmt %	Branch %	Cond %	Notes
round_robin.sv (DUT)	100%	100%	~95%	1 FEC condition missed
rr_driver.sv	100%	100%	100%	—
rr_monitor.sv	100%	100%	100%	—
rr_generator.sv	~69%	~69%	—	TYPE=2 paths partially exercised
rr_scoreboard.sv	~73%	~73%	—	TYPE=2 weight branches not fully hit
rr_transaction.sv	~0%	100%	100%	Class included via package; copy() not called

The single missed FEC (Functional Expression Coverage) condition in `round_robin.sv` corresponds to an edge case where all requesters assert simultaneously and all weights are exactly equal in TYPE=2 — a corner case that would require targeted directed testing to hit. All other DUT code paths are exercised by the random test.

7. Conclusion

This project successfully designed, implemented, and verified a parameterised round robin arbiter supporting three distinct arbitration policies: Conventional RR (TYPE=0), Modified RR (TYPE=1), and Weighted RR (TYPE=2). The DUT was verified across four configurations of N (4, 5, 7, 10) with 100 randomised transactions per run.

7.1 Key Findings

- **TYPE=0 and TYPE=1:** All 59 scoreboard checks passed (first transaction skipped due to the pipeline cycle) across all N values. The testbench correctly identifies the one-cycle latency of o_gnt and accounts for it via the prev/curr transaction scheme.
- **TYPE=2 DUT:** Waveform inspection confirms correct weighted arbitration behaviour. Weight counters load correctly from i_weights, decrement on each grant, and the highest-weight requester receives priority. The DUT is functionally correct.
- **TYPE=2 Scoreboard:** The reference model has a known timing bug — it applies the loaded weights to the current cycle rather than the next. This is a DUT defect.
- **Code Coverage:** The DUT achieved 100% statement and branch coverage. One FEC condition in TYPE=2 was not hit, corresponding to the equal-weight tie-breaking edge case.

7.2 Lessons Learned

- **Clocking block discipline:** Separating driver and monitor into distinct clocking blocks with appropriate skew values (#1 output, #1step input) is essential to prevent race conditions in batch simulation mode.
- **Registered output handling:** The DUT registers o_gnt, creating a one-cycle pipeline delay. The prev/curr transaction pairing in the scoreboard is required to correctly align requests with grants.
- **Package compilation:** Including all class files inside a single package eliminates double-compilation errors that arise when files are compiled both standalone and as part of a package.
- **Scoreboard timing for TYPE=2:** Weight registers in the DUT are non-blocking assignments — they take effect on the next posedge. Scoreboard reference models must mirror this timing exactly or false failures result.

7.3 Future Work

- Fix the TYPE=2 scoreboard weight-load timing so weight loads are applied to cycle N+1 rather than cycle N in the reference model.
 - Add a functional covergroup to verify that all N requester bits are granted at least once, all ptr values are visited, and en=0 idle cycles are exercised.
 - Extend the testbench to a full UVM environment with uvm_agent, uvm_sequencer, and uvm_coverage_collector components.
-

-
- Add directed test sequences targeting the known missed FEC condition (all equal weights, all requesters active).